# Esse ai resolve com CCW

Thomaz Miranda, Miguel Batista, Joao Arthur Marques

April 1, 2026

**Code Contents**


**1 Data Structures** 1
- 1.1 Fenwick Tree with Range Updates 1
- 1.2 Convex Hull Trick Dinamico 1
- 1.3 Disjoint Set Union (Union-Find) 1
- 1.4 DSU with Rollback (Persistent/Undoable Union-Find) 1
- 1.5 Segment Tree Over Time (Dynamic Connectivity Skeleton) 1
- 1.6 Fenwick Tree (Binary Indexed Tree) 2
- 1.7 Custom Hash for hash table 2
- 1.8 Mo's Algorithm (Offline Range Queries) 2
- 1.9 Order-Statistic Tree (PBDS) 2
- 1.10 Persistent Segment Tree - O(log n) per query/update 2
- 1.11 SegTree 2D Iterativa 3
- 1.12 Segment Tree (Range Query + Point Update) 3
- 1.13 Segment Tree (Range Query + Range Update) 3
- 1.14 Sparse Table (Idempotent Range Query) 4
- 1.15 Implicit Treap (Sequence Treap) 4

**2 Graph Algorithms** 4
- 2.1 2-SAT 4
- 2.2 Articulation Points / Bridges / Biconnected Components 5
- 2.3 Bellman-Ford Single-Source Shortest Paths 5
- 2.4 Centroid Decomposition - pares a distancia k 5
- 2.5 Centroid Tree 5
- 2.6 Dijkstra's Shortest Paths 6
- 2.7 Dinic's Maximum Flow (with Scaling) 6
- 2.8 Directed Minimum Spanning Tree (Edmonds / Chu-Liu) 6
- 2.9 Dominator Tree (Lengauer-Tarjan) 7
- 2.10 Euler Path / Euler Cycle 7
- 2.11 Floyd-Warshall Algorithm 7
- 2.12 Heavy Light Decomposition (Path query + Path update) 7
- 2.13 Hungarian algorithm — min-cost perfect matching 8
- 2.14 LCA com Euler tour + Sparse Table (RMQ) 8
- 2.15 LCA com binary lifting 8
- 2.16 Min Cost Max Flow (MCMF) 8
- 2.17 Strongly Connected Components (Kosaraju) 9
- 2.18 Topological Sort (Kahn's Algorithm) 9
- 2.19 Virtual Tree 9

**3 Mathematics** 9
- 3.1 Chinese Remainder Theorem 9
- 3.2 Extended Eucidian Algorithm 10
- 3.3 FFT/NTT Convolution 10
- 3.4 Divisor Decomposition / Square Root Trick 10
- 3.5 Gauss 10
- 3.6 Gauss - Z2 10
- 3.7 Flattened Divisor Sieve (CSR format) 11
- 3.8 Matrix template 11
- 3.9 Miller-Rabin Primality Test (Deterministic for 64-bit) 11
- 3.10 Modular Integer 11
- 3.11 Modular Arithmetic Helpers 12
- 3.12 Discrete Logarithm (Baby-step Giant-step) 12
- 3.13 Discrete Square Root 12
- 3.14 Primitive Root (Generator) 12
- 3.15 Quadratic Field Extension (Z_p[sqrt(D)]) 12
- 3.16 Linear Sieve (prime precomputation) 13

**4 String Algorithms** 13
- 4.1 Aho-Corasick Automaton 13
- 4.2 Knuth-Morris-Pratt (KMP) 13
- 4.3 Suffix Array - O(n log n) 13
- 4.4 Trie (Prefix Tree) 14
- 4.5 Z-funtion 14

**5 Dynamic Programming** 14
- 5.1 Divide and Conquer DP Optimization 14
- 5.2 Sum over Subsets DP (SOS DP) 14

**6 Computational Geometry** 14
- 6.1 Circumcircle and Circumradius 14
- 6.2 Convex Hull (Monotone Chain) 14
- 6.3 Extremal Vertex in Convex Polygon 15
- 6.4 Float Geometry Primitives 15
- 6.5 Integer Geometry Primitives 15
- 6.6 Halfplane Intersection 16
- 6.7 Point Inside Polygon 16
- 6.8 Point Inside Convex Polygon 16
- 6.9 Minimum Enclosing Circle 16
- 6.10 Minkowski Sum 17
- 6.11 Plota Poligonos 17
- 6.12 Polygon Cut 17
- 6.13 Segment Sweep Line Skeleton 17

**7 Extra (Templates and Utilities)** 17
- 7.1 1D Coordinate Compression 17
- 7.2 hash 17
- 7.3 makefile 17
- 7.4 Pragmas 17
- 7.5 random 17
- 7.6 stress 17
- 7.7 template 17
- 7.8 tester 18


**Theoretical Contents**


**1 Counting Problems** 18
- 1.1 Burnside's Lemma 18
- 1.2 Catalan Numbers 18

**2 Graphs** 18
- 2.1 Sum of Subtrees of a Tree 18

**3 Number Theory** 18
- 3.1 Chicken McNugget Theorem 18
- 3.2 Identities 18
- 3.3 Large Prime Gaps 19
- 3.4 Lucas's Theorem 19
- 3.5 Mobius Inversion 19
- 3.6 Number of Divisors 19
- 3.7 Prime Counting Function - $\pi(x)$ 19
- 3.8 Some Primes 19


# 1 Data Structures

## 1.1 Fenwick Tree with Range Updates 606

*Implements a pair of BITs to support 0-based range add updates and range sum queries efficiently.*

**Complexity:** O(log N) per op, O(N)

```
e87 class BIT{
446     ll bit[2][MAX+2];
5c1     ll n;
a76 public:
293     BIT(ll n2, v64& v) {
7a3         n = n2;
799         for (ll i = 1; i <= n; i++)
834             bit[1][min(n+1, i+(i&-i))] += bit[1][i] += v[i-1];
b17     }
3e6     ll get(ll x, ll i) {
125         ll ret = 0;
77f         for (; i; i -= i&-i) ret += bit[x][i];
3cb         return ret;
274     }
500     void add(ll x, ll i, ll val) {
2b2         for (; i <= n; i += i&-i) bit[x][i] += val;
249     }
a0f     ll get2(ll p) {
a6e         return get(0, p) * p + get(1, p);
c32     }
a76     ll query(ll l, ll r) {
897         return get2(r+1) - get2(l);
2fa     }
f55     void update(ll l, ll r, ll x) {
42e         add(0, l+1, x), add(0, r+2, -x);
f83         add(1, l+1, -x*l), add(1, r+2, x*(r+1));
b97     }
606 }
```

## 1.2 Convex Hull Trick Dinamico 66a

*para double, use INF = 1/.0, div(a, b) = a/b update(x) atualiza o ponto de intersecao da reta x overlap(x) verifica se a reta x sobrepoe a proxima add(a, b) adiciona reta da forma ax + b query(x) computa maximo de ax + b para entre as retas O(log(n)) amortizado por insercao O(log(n)) por query*

```
5c0 struct Line {
647     mutable ll a, b, p;
f15     bool operator<(const Line& o) const { return a < o.a; }
22f     bool operator<(ll x) const { return p < x; }
7f5 };

8a7 struct CHT : multiset<Line, less<>> {
9cb     ll div(ll a, ll b) {
941         return a / b - ((a ^ b) < 0 && a % b);
63f     }

956     void update(iterator x) {
c7b         if (next(x) == end()) x->p = INF;
36b         else if (x->a == next(x)->a) x->p = x->b >= next(x)->b ?
 INF : -INF;
558         else x->p = div(next(x)->b - x->b, x->a - next(x)->a);
d23     }

cf0     bool overlap(iterator x) {
9b4         update(x);
54b         if (next(x) == end()) return 0;
b2f         if (x->a == next(x)->a) return x->b >= next(x)->b;
2cd         return x->p >= next(x)->p;
136     }

1a7     void add(ll a, ll b) {
395         auto x = insert({a, b, 0});
67e         while (overlap(x)) erase(next(x)), update(x);
5f6         if (x != begin() && !overlap(prev(x))) x = prev(x),
update(x);
af3         while (x != begin() && overlap(prev(x)))
c8a             x = prev(x), erase(next(x)), update(x);
8e8     }
```

```
1f7     ll query(ll x) {
84a         assert(!empty());
1e0         auto l = *lower_bound(x);
8d4 #warning cuidado com overflow!
6a8         return l.a * x + l.b;
e51     }
287 };
```

## 1.3 Disjoint Set Union (Union-Find) 55e

*Supports find with path compression and union by size to maintain dynamic connectivity of disjoint sets.*

**Complexity:** O(alpha(N)) amortized per op, O(N)

```
da2 struct dsu {
c4c     v64 id, len;

b10
    dsu(ll n) : id(n), len(n, 1) { iota(id.begin(), id.end(), 0); }

15e
    ll find(ll a) { return a == id[a] ? a : id[a] = find(id[a]); }

847     bool uni(ll a, ll b) {
f78         a = find(a), b = find(b);
6c5         if (a == b) return false;
940         if (len[a] < len[b]) swap(a, b);
3b0         len[a] += len[b], id[b] = a;
01d         return true;
433     }
55e };
```

## 1.4 DSU with Rollback (Persistent/Undoable Union-Find) 8f1

*Allows merging sets and undoing operations to a previous state using a stack of modifications. No Path Compression to keep tree structure reversible.*

**Complexity:** O(log N) per find/unite, O(1) per checkpoint/rollback

```
86f struct dsu_rb {
c4c     v64 id, len;
7f8     ll allbip = true;
1c5     stack<pair<ll&, ll>> st;

798     dsu_rb(ll n) : id(n), len(n, 1) {
7a7         iota(id.begin(), id.end(), 0);
ce9     }

704     void save(ll &x) {  st.emplace(x, x);}

74d     ll time() { return sz(st); }
964     void rollback(ll t) {
358         while(sz(st) > t) {
6f2             auto [end, val] = st.top(); st.pop();
0d6             end = val;
e30         }
cb5     }

a27     ll find(ll a) { return a == id[a] ? a : find(id[a]); }

58e     bool unite(ll a, ll b) {
f78         a = find(a), b = find(b);
6c5         if (a == b) return false;
940         if (len[a] < len[b]) swap(a, b);
096         save(len[a]);
a2f         save(id[b]);
3b0         len[a] += len[b], id[b] = a;
01d         return true;
97d     }
8f1 };
```

## 1.5 Segment Tree Over Time (Dynamic Connectivity Skeleton) 3be

*Stores edge activation intervals in a segment tree over time to enable offline dynamic connectivity with rollback DSU.*

```
0e2 struct time_query{
c62     ll l, r;
b1b     time_query(ll l_, ll r_){
5a0         l = l_;
4d0         r = r_;
d69     }
0c7 };

44a struct time_node {
476     ll lm, rm;
504     unique_ptr<time_node> lc, rc;

88e     vector<time_query> op;

e4d     time_node(ll lm_, ll rm_){
b8a         lm = lm_;
f5b         rm = rm_;
b9e         if (lm != rm) {
ae7             ll mid = (lm + rm) / 2;
528             lc = make_unique<time_node>(lm, mid);
880             rc = make_unique<time_node>(mid + 1, rm);
d01         }
a0c     }

6df     void add_query(ll lq, ll rq, time_query x) {
7e8         if (rq < lm || lq > rm) return;
ca5         if (lq <= lm && rm <= rq) {
ddf             op.push_back(x);
e19             return;
4c3         }
d6b         lc->add_query(lq, rq, x);
8e3         rc->add_query(lq, rq, x);
95c     }
58a };
```

## 1.6 Fenwick Tree (Binary Indexed Tree) 590

*Supports point updates and prefix/range sum queries in logarithmic time using a 1-indexed BIT.*

**Complexity:** O(log N) per op, O(N)

```
e7f struct Bit {
5c1     ll n;
fa4     v64 bit;
1cd     Bit(ll _n = 0) : n(_n), bit(n + 1) {}
bd3     Bit(v64& v) : n(sz(v)), bit(n + 1) {
3a1         for (ll i = 1; i <= n; i++) {
289             bit[i] += v[i - 1];
481             ll j = i + (i & -i);
b3c             if (j <= n) bit[j] += bit[i];
631         }
e3d     }
adf     void update(ll i, ll x) { // soma x na posicao i
7fa         for (i++; i <= n; i += i & -i) bit[i] += x;
53a     }
8e6     ll pref(ll i) { // soma [0, i]
125         ll ret = 0;
8fb         for (i++; i; i -= i & -i) ret += bit[i];
3cb         return ret;
4bb     }
a76     ll query(ll l, ll r) {  // soma [l, r]
745         return pref(r) - pref(l - 1);
9a9     }
a32     ll upper_bound(ll x) {
fc8         ll p = 0;
3b6         for (ll i = __lg(n); i+1; i--)
086             if (p + (1<<i) <= n && bit[p + (1<<i)] <= x)
218                 x -= bit[p += (1 << i)];
4e5         return p;
c1c     }
590 };
```

## 1.7 Custom Hash for hash table 917

*Provides 64-bit hashers for integers and pairs to use with __gnu_pbds::gp_hash_table.*

**Complexity:** O(1) average, O(n)

```
12f #include <bits/extc++.h>

// for ll
c2a struct chash {
445     const uint64_t C = ll(4e18 * acos(0)) | 71;
a0e     ll operator()(ll x) const { return __builtin_bswap64(x*C); }
615 };

// for p64
c2a struct chash {
81a     size_t operator()(const p64& p) const {
9da         return p.first ^ __builtin_bswap64(p.second);
152     }
ad8 };

666 __gnu_pbds::gp_hash_table<ll, ll, chash> h({},{},{},{},{1<<16});
e50
__gnu_pbds::gp_hash_table<p64, ll, chash> h({},{},{},{},{1<<16});
```

## 1.8 Mo's Algorithm (Offline Range Queries) 694

*Answers offline range queries by ordering them (block or Hilbert curve) to get small pointer movement and amortized updates.*

**Complexity:** O((N + Q) sqrt N), O(N)

```
82a const ll MAX = 2e5+10;
b4b const ll SQ = sqrt(MAX);
432 ll ans;

0a7 inline void insert(ll p) {
651 }

709 inline void erase(ll p) {
f63 }

6b5 inline ll hilbert(ll x, ll y) {
602     static ll N = 1 << (__builtin_clzll(0ll) - __builtin_clzll(
MAX));
a12     ll rx, ry, s;
60f     ll d = 0;
d33     for (s = N/2; s > 0; s /= 2) {
```

```
357         rx = (x & s) > 0, ry = (y & s) > 0;
89c         d += s * ll(s) * ((3 * rx) ^ ry);
67e         if (ry == 0) {
50d             if (rx == 1) x = N-1 - x, y = N-1 - y;
f50             swap(x, y);
f01         }
909     }
fc9     return d;
f00 }

b99 #define HILBERT true
55e vector<ll> MO(vector<pair<ll, ll>> &q) {
36d     ans = 0;
2c6     ll m = sz(q);
0f3     vector<ll> ord(m);
5be     iota(ord.begin(), ord.end(), 0);
6a6 #if HILBERT
f82     vector<ll> h(m);
96d     for (ll i = 0; i < m; i++) h[i] = hilbert(q[i].first, q[i].
second);
697     sort(ord.begin(), ord.end(), [&](ll l, ll r) { return h[l] <
 h[r]; });
8c1 #else
d64     sort(ord.begin(), ord.end(), [&](ll l, ll r) {
4a5         if (q[l].first / SQ != q[r].first / SQ) return q[l].
first < q[r].first;
816
        if ((q[l].first / SQ) % 2) return q[l].second > q[r].second;
ae2         return q[l].second < q[r].second;
69e     });
f2e #endif
6ae     vector<ll> ret(m);
8f0     ll l = 0, r = -1;

0ac     for (ll i : ord) {
f0a         ll ql, qr;
d68         tie(ql, qr) = q[i];
fa8         while (r < qr) insert(++r);
a8e         while (l > ql) insert(--l);
23e         while (l < ql) erase(l++);
46a         while (r > qr) erase(r--);
29c         ret[i] = ans;
272     }
3cb     return ret;
7a8 }
```

## 1.9 Order-Statistic Tree (PBDS) 468

*Wraps __gnu_pbds tree to support order_of_key and find_by_order operations on a sorted set.*

**Complexity:** O(log N) per op, O(N)

```
663 #include <ext/pb_ds/assoc_container.hpp>
8ea #include <ext/pb_ds/tree_policy.hpp>

df9 using namespace __gnu_pbds;

99c #define ordered_set tree<p64, null_type,less<p64>, rb_tree_tag,
tree_order_statistics_node_update>

8c2 int main() {
fe6     ordered_set s;
dbe     s.find_by_order(position);
ab6     s.order_of_key(value);
731 }
```

## 1.10 Persistent Segment Tree - O(log n) per query/update e2c

*Supports point updates and range queries across versions*

```
cd2 struct Node {
091     ll val;
c62     ll l, r;
7b0 };

ab6 struct perseg {
031     ll n, cnt = 0;
c8e     vector<Node> nd;
619     v64 rt;

26c     perseg(v64& v, ll q = 0) : n(sz(v)) {
a66         nd.reserve(2*n + 18*(n+q));
180         rt.reserve(q + 2);
49d         rt.push_back(cnt++);
ed5         build(nd.data(), cnt, rt[0], 0, n-1, v);
721     }
442
    static ll build(Node* d, ll& cnt, ll p, ll l, ll r, v64& v) {
7ad         if (l == r) return d[p].val = v[l];
ed8         d[p].l = cnt++; d[p].r = cnt++;
c61         ll m = (l+r)/2;
7c0         return d[p].val = build(d, cnt, d[p].l, l, m, v) + build
(d, cnt, d[p].r, m+1, r, v);
739     }
16c     ll query(ll a, ll b, ll t) {
c37         return query(nd.data(), a, b, rt[t], 0, n-1);
8a7     }
726     static ll query(Node* d, ll a, ll b, ll p, ll l, ll r) {
ee3         if (b < l || r < a) return 0;
027         if (a <= l && r <= b) return d[p].val;
c61         ll m = (l+r)/2;
af0         return query(d, a, b, d[p].l, l, m) + query(d, a, b, d[p
].r, m+1, r);
053     }
114     ll set(ll a, ll val, ll t) {
946         ll nr = cnt++;
cb5         set(nd.data(), cnt, a, val, rt[t], nr, 0, n-1);
329         rt.push_back(nr);
b87         return (ll)rt.size()-1;
3fe     }
a20     static ll set(Node* d, ll& cnt, ll a, ll val, ll lp, ll p,
ll l, ll r) {
59f         if (l == r) return d[p].val = val;
c61         ll m = (l+r)/2;
ca8         if (a <= m) {
44b             d[p].l = cnt++; d[p].r = d[lp].r;
d20             return d[p].val = set(d, cnt, a, val, d[lp].l, d[p].
l, l, m) + d[d[p].r].val;
d13         }
49e         d[p].l = d[lp].l; d[p].r = cnt++;
f1d         return d[p].val = d[d[p].l].val + set(d, cnt, a, val, d[
lp].r, d[p].r, m+1, r);
70b     }
ccf };
```

## 1.11 SegTree 2D Iterativa 2c9

*Consultas 0-based Um valor inicial em (x, y) deve ser colocado em seg[x+n][y+n] Query: soma do retangulo ((x1, y1), (x2, y2)) Update: muda o valor da posicao (x, y) para val Nao pergunte como que essa coisa funciona Para query com distancia de manhattan <= d, faca nx = x+y, ny = x-y Update em (nx, ny), query em ((nx-d, ny-d), (nx+d, ny+d)) Se for de min/max, pode tirar os if's da 'query', e fazer sempre as 4 operacoes. Fica mais rapido Complexidades: build - O(n{}2) query - O(log{}2(n)) update - O(log{}2(n))*

```
159 struct seg2d{
5c1     ll n;
224     vector<v64> seg;

529     seg2d(ll n_, vector<v64>& init) : n(n_), seg(2 * (n+1),
v64(2 * (n+1), 0)) {
15b         forn(i,0,n) forn(j,0,n)
aaf             seg[i + n][j + n] = init[i][j];
67d         for (ll x = 2*n; x; x--) for (ll y = 2*n; y; y--) {
1d9             if (x < n) seg[x][y] = seg[2*x][y] + seg[2*x+1][y];
7ea             if (y < n) seg[x][y] = seg[x][2*y] + seg[x][2*y+1];
8b8         }
11e     }

19c     ll query(ll x1, ll y1, ll x2, ll y2) {
d57         ll ret = 0, y3 = y1 + n, y4 = y2 + n;
903         for (x1 += n, x2 += n; x1 <= x2; ++x1 /= 2, --x2 /= 2)
07a         for (y1 = y3, y2 = y4; y1 <= y2; ++y1 /= 2, --y2 /= 2) {
91e             if (x1%2 == 1 && y1%2 == 1) ret += seg[x1][y1];
106             if (x1%2 == 1 && y2%2 == 0) ret += seg[x1][y2];
8b2             if (x2%2 == 0 && y1%2 == 1) ret += seg[x2][y1];
9dc             if (x2%2 == 0 && y2%2 == 0) ret += seg[x2][y2];
134         }
3cb         return ret;
366     }

5f5     void update(ll x, ll y, ll val) {
873         ll y2 = y += n;
559         for (x += n; x; x /= 2, y = y2) {
5f1             if (x >= n) seg[x][y] = val;
559             else seg[x][y] = seg[2*x][y] + seg[2*x+1][y];

8a4
            while (y /= 2) seg[x][y] = seg[x][2*y] + seg[x][2*y+1];
511         }
f51     }
2c9 };
```

## 1.12 Segment Tree (Range Query + Point Update) de6

*supports point updates and range queries.*

**Complexity:** O(log N) per op, O(N)

```
5a4 struct node {
42e     ll val = 0;

c31     static node comb(const node& a, const node& b) {
5b6         return {a.val + b.val};
e5f     }
b99 };

52f template<typename T> struct segtree {
5c1   ll n;
141   T neutral;
8fd   vector<T> tree;
```

```
6f5   segtree(ll _n, T _neutral = T()) {
384     n = _n, neutral = _neutral;
19d     tree.resize(2*n+1, neutral);
411   }

ab6   void set_leaves(vector<T> &leaves) {
ba7     copy(leaves.begin(), leaves.end(), tree.begin() + n);

da6     for (ll i = n - 1; i > 0; i--) tree[i] = T::comb(tree[2 * i],
 tree[2 * i + 1]);
1c0   }

a99   void update(ll i, T v) {
2c1     i += n;
440     tree[i] = v;
9fc     while (i > 1) {
b73       i /= 2;
a5b       tree[i] = T::comb(tree[2 * i], tree[2 * i + 1]);
cbd     }
9be   }

894   T query(ll i, ll j) {
bf2     T rl = neutral, rr = neutral;
2b8     for(i += n, j += n; i <= j; i /= 2, j /= 2){
40f       if((i&1) == 1) rl = T::comb(rl, tree[i++]);
0e9       if((j&1) == 0) rr = T::comb(tree[j--], rr);
933     }
5e7     return T::comb(rl, rr);
d69   }
a85 };
```

## 1.13 Segment Tree (Range Query + Range Update) 8e0

*Tree for range queries with a customizable combine; supports range updates and range queries.*

**Complexity:** O(log N) per op, O(N) to build

```
cd2 struct Node {
cbd     ll sum, cnt;
54d     Node operator*(const Node &o) const { return {sum + o.sum,
cnt + o.cnt}; }
135 };

9f2 struct Update {
23e     ll add = 0;
997     optional<ll> set;

02c     Node operator()(const Node &n) const {
13d         ll res = set.has_value() ? *set * n.cnt : n.sum;
4c9         return {res + add * n.cnt, n.cnt};
9d0     }

919     Update operator+(const Update &o) const {
5f4         Update res = *this;
a36         if (o.set.has_value()) {
f1e             res.set = o.set;
9f5             res.add = 0;
18c         }
7fe         if (o.add != 0) {
fa9             if (res.set.has_value()) *res.set += o.add;
63b             else res.add += o.add;
e1c         }
8df         return res;
407     }
dad };

58d template<typename T, typename U> struct segtree {
471   ll s, h;

808   T id;
377   vector<T> val;

a3e   U noop;
d47   vector<bool> dirty;
65b   vector<U> prop;

a57   segtree(ll ts, T tid = T(), U tnoop = U()) {
11b     id = tid, noop = tnoop;
836     for (s = 1, h = 1; s < ts; ) s *= 2, h++;

8e7     val.assign(2*s, id);
4a5     dirty.assign(2*s, false);
363     prop.assign(2*s, noop);
889   }

6db   void set_leaves(vector<T> &lvs) {
a5c     copy(lvs.begin(), lvs.end(), val.begin() + s);

95f     for (ll i = s - 1; i > 0; i--) val[i] = val[2 * i] * val[2 *
 i + 1];
4a5     dirty.assign(2*s, false);
363     prop.assign(2*s, noop);
3a5   }

570   void apply(ll i, U &upd) {
564     val[i] = upd(val[i]);
c4c     if(i < s) {
675       prop[i] = prop[i] + upd;
2d8       dirty[i] = true;
2ca     }
703   }

30b   void pull(ll i) {
fa9     for (ll l = i/2; l; l /= 2) {
c7a       T comb = val[2*l] * val[2*l+1];
a08       val[l] = prop[l](comb);
325     }
eee   }

540   void push(ll i) {
c2d     for (ll th = h; th > 0; th--) {
dcd       ll l = i >> th;

38a       if (dirty[l]) {
4fb         apply(2*l, prop[l]);
fd5         apply(2*l+1, prop[l]);

6ad         prop[l] = noop;
7b3         dirty[l] = false;
983       }
db3     }
7b6   }

41b   void update(ll i, ll j, U upd) {
33a     i += s, j += s;
f9d     push(i), push(j);

2a7     for (ll l = i, r = j; l <= r; l /= 2, r /= 2) {
6a1       if((l&1) == 1) apply(l++, upd);
7a4       if((r&1) == 0) apply(r--, upd);
f70     }

cf2     pull(i), pull(j);
470   }

894   T query(ll i, ll j){
33a     i += s, j += s;
f9d     push(i), push(j);

487     T rl = id, rr = id;
810     for(; i <= j; i /= 2, j /= 2){
2bc       if((i&1) == 1) rl = rl * val[i++];
03c       if((j&1) == 0) rr = val[j--] * rr;
4a8     }
f0e     return rl * rr;
b69   }
429 };
```

## 1.14 Sparse Table (Idempotent Range Query) 191

*Preprocesses static array to answer idempotent range queries (e.g., min/max) in O(1) after O(N log N) build.*

**Complexity:** O(N log N) build, O(1) query; O(N log N) space

```
549 struct sparse {
d87     vector<v64> m;

7bf     sparse(v64& v) {
3d3         ll n = sz(v);
571         ll logn = 64 - __builtin_clzll(n - 1);
260         m.resize(n+1, v64(logn+1));

725         forn(i, 0, n) m[i][0] = v[i];

2eb         for (ll j = 1; (1 << j) <= n; j++)
f0e         for (ll i = 0; i + (1 << j) <= n; i++)
4fe             m[i][j] = min(m[i][j-1], m[i + (1 << (j-1))][j-1]);
780     }

cb2     ll query(ll a, ll b) {
19f         ll j = __builtin_clzll(1) - __builtin_clzll(b - a + 1);
966         return min(m[a][j], m[b - (1 << j) + 1][j]);
525     }
191 };
```

## 1.15 Implicit Treap (Sequence Treap) 0a3

*Maintains a sequence with split and merge operations using randomized priorities and subtree sizes.*

**Complexity:** O(log N) expected per op, O(N)

```
bcd struct Treap{
091     ll val;
549     ll prio, size;
b09     vector<Treap*> kids;
e02     Treap(ll c): val(c), prio(rand()), size(1),
bc6         kids({NULL,NULL}){};
2f4 };

0c2 ll size(Treap *me){return me ? me->size : 0;}
ae2 void rsz(Treap* me){me -> size =
```

```
5bf     1 + size(me->kids[0]) + size(me->kids[1]);}

5d0 vector<Treap*> split(Treap *me, ll idx){
79a     if(!me) return {NULL,NULL};
161     vector<Treap*> out;

9b0     if(size(me->kids[0]) < idx){
863         auto aux = split(me->kids[1],
4b0             idx - size(me->kids[0]) -1);
58e         me->kids[1] = aux[0];
3b0         rsz(me);
853         out = {me, aux[1]};
8cc     }else{
72d         auto aux = split(me->kids[0], idx);
f69         me->kids[0] = aux[1];
3b0         rsz(me);
d01         out = {aux[0], me};
26e     }
b44     return out;
683 }

a3f Treap* merge(Treap *left, Treap *right){
b53     if(left == NULL) return right;
03c     if(right == NULL) return left;

b29     Treap* out;

994     if(left->prio < right->prio){
1e6         left->kids[1] = merge(left->kids[1], right);
a81         rsz(left);
3a5         out = left;
b7a     }else{
5e8         right->kids[0] = merge(left, right->kids[0]);
4c9         rsz(right);
93a         out = right;
2d7     }
b44     return out;
629 }
```

# 2 Graph Algorithms

## 2.1 2-SAT ed6

*solve() retorna um par, o first fala se eh possivel atribuir, o second fala se cada variavel eh verdadeira O(|V|+|E|) = O(#variaveis + #restricoes)*

```
ecf struct sat {
7fa     ll n, tot;
cf5     vector<v64> g;
83e     v64 vis, comp, id, ans;
b07     v64 s;

6d2     sat() {}
3c5     sat(ll n_) : n(n_), tot(n), g(2*n) {}

b9d     ll dfs(ll i, ll& t) {
ad4         ll lo = id[i] = t++;
09c         s.push_back(i), vis[i] = 2;
2b0         for (ll j : g[i]) {
4e4             if (!vis[j]) lo = min(lo, dfs(j, t));
840             else if (vis[j] == 2) lo = min(lo, id[j]);
228         }
b84         if (lo == id[i]) while (1) {
481             ll u = s.back(); s.pop_back();
7ff             vis[u] = 1, comp[u] = i;
506             if ((u>>1) < n && ans[u>>1] == -1) ans[u>>1] = ~u&1;
56b             if (u == i) break;
969         }
ad1         return lo;
708     }

5d5     void add_impl(ll x, ll y) { // x -> y = !x ou y
84f         x = x >= 0 ? 2*x : -2*x-1;
dc6         y = y >= 0 ? 2*y : -2*y-1;
ef5         g[x].push_back(y);
add         g[y^1].push_back(x^1);
b3c     }
e62     void add_cl(ll x, ll y) { // x ou y
935         add_impl(~x, y);
64b     }
00b     void add_xor(ll x, ll y) { // x xor y
8ab         add_cl(x, y), add_cl(~x, ~y);
ace     }
3f7     void add_eq(ll x, ll y) { // x = y
a3b         add_xor(~x, y);
68c     }
dbb     void add_true(ll x) { // x = T
956         add_impl(~x, x);
564     }
dbf     void at_most_one(v64 v) { // no max um verdadeiro
909         g.resize(2*(tot+sz(v)));
9fc         for (ll i = 0; i < sz(v); i++) {
33c             add_impl(tot+i, ~v[i]);
c82             if (i) {
a6f                 add_impl(tot+i, tot+i-1);
d24                 add_impl(v[i], tot+i-1);
586             }
7dc         }
136         tot += sz(v);
1a0     }

d26     pair<bool, v64> solve() {
f59         ans = v64(n, -1);
b4d         ll t = 0;
d68         vis = comp = id = v64(2*tot, 0);
96e         for (ll i = 0; i < 2*tot; i++) if (!vis[i]) dfs(i, t);
289         for (ll i = 0; i < tot; i++)
105             if (comp[2*i] == comp[2*i+1]) return {false, {}};
d78         return {true, ans};
8d8     }
ed6 };
```

## 2.2 Articulation Points / Bridges / Biconnected Components da9

*is_art[i] = true se i eh ponto de articulacao (ART) bridges: lista de arestas ponte (BRIDGE) bcc: lista de componentes biconexas (em arestas) (BCC) O(n+m)*

```
2b1 struct TJ {
a12     ll n, timer=0;
295     vector<v64>& g;
004     v64 tin, low;

8c9     vector<bool> is_art; // ART
eed     vector<p64> bridges; // BRIDGE
62a     vector<p64> st; // BCC
0aa     vector<vector<p64>> bcc; // BCC

53b     TJ(ll n, vector<v64>& g): n(n), g(g){
5fa         tin.assign(n,-1);
ca6         low.resize(n);
cdd         is_art.assign(n, false); // ART
daa         forn(i, 0, n) if(tin[i]==-1) dfs(i);
b0f     }

882     void dfs(ll v, ll p=-1){
405         tin[v]=low[v]=timer++;
92d         ll ch=0;

5f4         for(ll to:g[v]){
0d5             if(to==p) continue;

483             if(tin[to]==-1){
fd9                 st.push_back({v,to}); // BCC

5b6                 dfs(to,v);
184                 low[v] = min(low[v], low[to]);

28d                 if(low[to]>tin[v]) bridges.push_back({v,to}); //
 BRIDGE
7bb                 if(low[to]>=tin[v] && p!=-1) is_art[v] = true; //
 ART

0a3                 if(low[to]>=tin[v]){ // BCC
bef                   vector<p64> c; // BCC
0db                   while(1){ // BCC
7ad                     auto e=st.back(); st.pop_back(); // BCC
58e                     c.push_back(e); // BCC
1a7                     if(e.first==v && e.second==to) break; // BCC
f9e                   } // BCC
36b                   bcc.push_back(c); // BCC
111                 } // BCC

d78                 ch++;
169             }else{
1ba                 low[v]=min(low[v],tin[to]);
a5e
                if(tin[to]<tin[v]) st.push_back({v,to});   // BCC
5d5             }
323         }
020         if(p==-1 && ch>1) is_art[v] = true;   // ART
97e     }
da9 };
```

## 2.3 Bellman-Ford Single-Source Shortest Paths 245

*Computes shortest paths from s on directed graphs with possibly negative edge weights. Nodes reachable via a negative cycle get dist = -INF.*

**Complexity:** O(N*M), O(N + M)

```
52b void bellman_ford(ll s, ll n, v64& d, v64& p, vector<tuple<ll,ll,
ll>> edges) {
7c5     d.assign(n, INF);
c87     p.assign(n, -1);
480     d[s] = 0;

ec2     sort(edges.begin(), edges.end(), [](auto& a, auto& b) {
8f0         ll sa = get<0>(a) < get<1>(a) ? get<0>(a) : -get<0>(a);
fa1         ll sb = get<0>(b) < get<1>(b) ? get<0>(b) : -get<0>(b);
76f         return sa < sb;
119     });

306     ll lim = n / 2 + 2; // use n-1 with unsorted edges
```

```
829     forn(i, 0, lim) for (auto [u, v, w] : edges) {
671         if (d[u] == INF || d[u] == -INF) continue;
4a6         ll nd = d[u] + w;
6be         if (nd < d[v]) {
1ca             p[v] = u;
c00             d[v] = (i < lim - 1 ? nd : -INF);
8bd         }
e00     }
361     forn(i, 0, lim) for (auto [u, v, w] : edges)
51c         if (d[u] == -INF) d[v] = -INF;
245 }
```

## 2.4 Centroid Decomposition - pares a distancia k 362

*Objetivo: contar #pares (u,v) com dist(u,v)=k na arvore. Ideia: em cada centroide c, para cada subarvore j: - path = profundidades d partindo de j (sem nos removidos) - somar cnt[k-d-1] ANTES de atualizar (evita dupla contagem entre subarvores) - depois, cnt[d+1]++ (a aresta (c,j) soma 1) Funcoes: dfs_sz (tamanhos), centroid (acha centroide), dfs (coleta depths), decomp (divide e combina). Complexidade: O(n log n) tempo; O(n) memoria. Cuidados: limites de cnt, k==0 trata a parte se quiser contar (u,u); rem[c] permanece marcado. escrito pelo gpt, cuidado*

```
c17 const ll MAX = 100'005;
978 vector<ll> g[MAX];
a44 ll siz[MAX], rem[MAX];

32f void dfs(v64& path, ll i, ll l=-1, ll d=0) {
bc8     path.push_back(d);
ca6
    for (ll j : g[i]) if (j != l && !rem[j]) dfs(path, j, i, d+1);
510 }

651 ll dfs_sz(ll i, ll l=-1) {
382     siz[i] = 1;
4a3
    for (ll j : g[i]) if (j != l && !rem[j]) siz[i] += dfs_sz(j, i);
b77     return siz[i];
459 }

054 ll centroid(ll i, ll l, ll size) {
880
    for (ll j : g[i]) if (j != l && !rem[j] && siz[j] > size / 2)
9a5         return centroid(j, i, size);
b36     return i;
19c }

23e ll decomp(ll i, ll k) {
87f     ll c = centroid(i, i, dfs_sz(i));
437     rem[c] = 1;

     // gasta O(n) aqui - dfs sem ir pros caras removidos
0a3     ll ans = 0;
38b     v64 cnt(siz[i]);
5fb     cnt[0] = 1;
e27     for (ll j : g[c]) if (!rem[j]) {
83d         v64 path;
70a         dfs(path, j);
f7e         for (ll d : path) if (0 <= k-d-1 && k-d-1 < siz[i])
1fc             ans += cnt[k-d-1];
bc0         for (ll d : path) cnt[d+1]++;
99e     }

244     for (ll j : g[c]) if (!rem[j]) ans += decomp(j, k);
694     rem[c] = 0;
ada     return ans;
cba }
```

## 2.5 Centroid Tree 445

*Constroi a centroid tree p[i] eh o pai de i na centroid-tree dist[i][k] = distancia na arvore original entre i e o k-esimo ancestral na arvore da centroid O(n log(n)) de tempo e memoria*

```
c17 const ll MAX = 100'005;
bec vector<v64> g(MAX), dist(MAX);
171 v64 siz(MAX), rem(MAX), p(MAX);

651 ll dfs_sz(ll i, ll l=-1) {
382     siz[i] = 1;
4a3
    for (ll j : g[i]) if (j != l && !rem[j]) siz[i] += dfs_sz(j, i);
b77     return siz[i];
459 }

054 ll centroid(ll i, ll l, ll size) {
880
    for (ll j : g[i]) if (j != l && !rem[j] && siz[j] > size / 2)
9a5         return centroid(j, i, size);
b36     return i;
19c }

72c void dfs_dist(ll i, ll l, ll d=0) {
ad5     dist[i].push_back(d);
ad1     for (ll j : g[i]) if (j != l && !rem[j])
4e6         dfs_dist(j, i, d+1);
561 }

6e2 void decomp(ll i, ll l = -1) {
87f     ll c = centroid(i, i, dfs_sz(i));
fca     rem[c] = 1, p[c] = l;
46b     dfs_dist(c, c);
13a     for (ll j : g[c]) if (!rem[j]) decomp(j, c);
52b }

c5b void build(ll n) {
76b     forn(i,0,n) rem[i] = 0, dist[i].clear();
6e7     decomp(0);
807     forn(i,0,n) reverse(dist[i].begin(), dist[i].end());
5f4 }
```

## 2.6 Dijkstra's Shortest Paths 6cd

*Computes single-source shortest paths on non-negative weighted graphs using a priority queue.*

**Complexity:** O((N + M) log N), O(N + M)

```
e1c void dijkstra(ll s, v64 &d, v64& p, vector<vector<p64>>& g) {
7ed     ll n = sz(g);
7c5     d.assign(n, INF);
c87     p.assign(n, -1);

480     d[s] = 0;
d8d     priority_queue<p64> pq;
33e     pq.push({0, s});
09f     while (!pq.empty()) {
27e         ll u = pq.top().second;
311         ll d_u = -pq.top().first;
6a5         pq.pop();
46d         if (d_u != d[u]) continue;

667         for (auto [v, w_v] : g[u]) {
f4f             if (d[u] + w_v < d[v]) {
0eb                 d[v] = d[u] + w_v;
1ca                 p[v] = u;
305                 pq.push({-d[v], v});
f3f             }
79c         }
bea     }
6cd }
```

## 2.7 Dinic's Maximum Flow (with Scaling) b41

*Computes max flow using Dinic's algorithm with optional capacity scaling to speed up BFS levels.*

**Complexity:** O(E V$\hat{\}$2) worst-case, O(E)

```
7c5 struct dinitz {
8df     const bool scaling;
16e     ll lim;
60e     struct edge {
914         ll to, cap, rev, flow;
1c9         bool res;
6ff         edge(ll to_, ll cap_, ll rev_, bool res_)
973
            : to(to_), cap(cap_), rev(rev_), flow(0), res(res_) {}
4d0     };

b0e     vector<vector<edge>> g;
c60     v64 lev, beg;
f30     ll F;
377
    dinitz(ll n, bool scaling_) : scaling(scaling_), g(n), F(0){}

fd8     void add(ll a, ll b, ll c) {
1ae         g[a].emplace_back(b, c, sz(g[b]), false);
3be         g[b].emplace_back(a, 0, sz(g[a])-1, true);
b9c     }

c3f     bool bfs(ll s, ll t) {
09a         lev = v64(sz(g), -1); lev[s] = 0;
f7f         beg = v64(sz(g), 0);
71c         queue<ll> q; q.push(s);
dad         while (sz(q)) {
9c6             ll u = q.front(); q.pop();
7af             for (auto& i : g[u]) {
099
                if (lev[i.to] != -1 || (i.flow == i.cap)) continue;
c7b                 if (scaling && i.cap - i.flow < lim) continue;
d86                 lev[i.to] = lev[u] + 1;
138                 q.push(i.to);
52f             }
c49         }
498         return lev[t] != -1;
544     }

d3c     ll dfs(ll v, ll t, ll f = INF) {
d24         if (!f || v == t) return f;
be3         for (ll& i = beg[v]; i < sz(g[v]); i++) {
29f             auto& e = g[v][i];
9dd             if (lev[e.to] != lev[v] + 1) continue;
e65             ll foi = dfs(e.to, t, min(f, e.cap - e.flow));
789             if (!foi) continue;
```

```
f4f             e.flow += foi, g[e.to][e.rev].flow -= foi;
a79             return foi;
d9e         }
ca4         return 0;
ad6     }

f34     ll max_flow(ll s, ll t) {
313         for (lim = scaling ? (1ll<<30) : 1; lim; lim /= 2)
3e3             while (bfs(s, t)) while (ll ff = dfs(s, t)) F += ff;
1fa         return F;
26f     }

2c0     vector<p64> min_cut() {
999         vector<p64> cut;
808         forn(u, 0, sz(g))
5db             if (lev[u] != -1)
b95                 for (auto& e : g[u])
f1a                     if (lev[e.to] == -1 && !e.res)
668                         cut.push_back({u, e.to});
03f         return cut;
c44     }

0e5     void reset() {
19b         F = 0;
74a         for (auto& edges : g) for (auto& e : edges) e.flow = 0;
50f     }
b41 };
```

## 2.8 Directed Minimum Spanning Tree (Edmonds / Chu-Liu) 3f5

*finds the minimum cost arborescence rooted at src in a weighted directed graph. returns {cost, parent} where parent[u] is the predecessor of u in the optimal tree. returns {-1, {}} if no arborescence exists.*

**Complexity:** O(M log N), O(N+M) me

```
43b #include "src/structures/dsu_rb.cpp"

720 struct Edge {ll a, b, w; };
cd2 struct Node {
453     Edge key;
fe6     Node *l, *r;
fa5     ll delta;
3b1     void prop() {
777         key.w += delta;
a45         if (l) l->delta += delta;
833         if (r) r->delta += delta;
389         delta = 0;
ebb     }
c8b     Edge top() { prop(); return key; }
4bc };
28b Node *merge(Node *a, Node *b) {
1d1     if (!a || !b) return a ?: b;
068     a->prop(), b->prop();
b8c     if (a->key.w > b->key.w) swap(a, b);
da8     swap(a->l, (a->r = merge(b, a->r)));
128     return a;
9ca }
f3e void pop(Node*& a) { a->prop(); a = merge(a->l, a->r); }

d7a pair<ll, v64> dmst(ll n, ll src, vector<Edge>& g) {
033     dsu_rb uf(n);
da5     vector<Node*> heap(n);
e8f     for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node{e});
2c5     ll res = 0;
441     v64 seen(n, -1), path(n), par(n);
1d1     seen[src] = src;
e0c     vector<Edge> Q(n), in(n, {-1,-1}), comp;
514     deque<tuple<ll, ll, vector<Edge>>> cycs;
b66     forn(s,0,n) {
df0         ll u = s, qi = 0, w;
aab         while (seen[u] < 0) {
6f9             if (!heap[u]) return {-1,{}};
e2f             Edge e = heap[u]->top();
e62             heap[u]->delta -= e.w, pop(heap[u]);
39f             Q[qi] = e, path[qi++] = u, seen[u] = s;
72b             res += e.w, u = uf.find(e.a);
80a             if (seen[u] == s) { /// found cycle, contract
70d                 Node* cyc = 0;
6da                 ll end = qi, time = uf.time();
3c0                 do cyc = merge(cyc, heap[w = path[--qi]]);
8c0                 while (uf.unite(u, w));
bd1                 u = uf.find(u), heap[u] = cyc, seen[u] = -1;
3d1                 cycs.push_front({u, time, {&Q[qi], &Q[end]}});
a31             }
89b         }
b87         forn(i,0,qi) in[uf.find(Q[i].b)] = Q[i];
468     }

b51     for (auto& [u,t,com] : cycs) { // restore sol (optional)
2a6         uf.rollback(t);
77b         Edge inEdge = in[u];
468         for (auto& e : com) in[uf.find(e.b)] = e;
026         in[uf.find(inEdge.b)] = inEdge;
177     }
446     forn(i,0,n) par[i] = in[i].a;
ea7     return {res, par};
d4f }
```

## 2.9 Dominator Tree (Lengauer-Tarjan) 142

*builds the dominator tree given a directed graph and a source. returns a vector where ret[u] is the immediate dominator of u.*

**Complexity:** O((N+M) log N), O(N+M) mem

```
9af v64 DomTree(vector<v64>& graph, ll src) {
9bc     ll n = sz(graph);
462     vector<v64> tree(n), trans(n), buck(n);
c65
    v64 semi(n), par(n), dom(n), label(n), atob(n, -1), btoa(n, -1),
bb6         link(n, -1);

423     function<ll(ll, ll)> find=[&](ll u, ll d){
671         if(link[u] == -1) return d ? -1 : u;
f60         ll v = find(link[u], d + 1);
ac3         if(v < 0) return u;
11c         if(semi[label[link[u]]] < semi[label[u]]) label[u] =
label[link[u]];
6d3         link[u] = v;
5ba         return d ? v : label[u];
89f     };
b4d     ll t = 0;
5bd     function<void(ll)> dfs=[&](ll u){
3bb         atob[u] = t;
539         btoa[t] = u;
c31         label[t] = semi[t] = t;
32c         t++;
ac5         for(auto v : graph[u]){
b23         if(atob[v] == -1) dfs(v), par[atob[v]] = atob[u];
4b7         trans[atob[v]].push_back(atob[u]);
28b         }
b24     };
295     dfs(src);
1da     for(ll u = t - 1; u >= 0; --u){
b73         for(auto v : trans[u]) semi[u] = min(semi[u], semi[find(
v, 0)]);
908         if(u) buck[semi[u]].push_back(u);
163         for(auto w : buck[u]){
556             ll v = find(w, 0);
fc8             dom[w] = semi[v] == semi[w] ? semi[w] : v;
542         }
66b         if(u) link[u] = par[u];
e9b     }
a0d     v64 ret(n, -1);
029     forn(u,1,t) {
453         if(dom[u] != semi[u]) dom[u] = dom[dom[u]];
0cc         ret[btoa[u]] = btoa[dom[u]];
d3e     }
3cb     return ret;
142 }
```

## 2.10 Euler Path / Euler Cycle 2f4

*Para declarar: 'euler<true> E(n);' se quiser direcionado e com 'n' vertices As funcoes retornam um par com um booleano indicando se possui o cycle/path que voce pediu, e um vector de {vertice, id da aresta para chegar no vertice} Se for get_path, na primeira posicao o id vai ser -1 get_path(src) tenta achar um caminho ou ciclo euleriano comecando no vertice 'src'. Se achar um ciclo, o primeiro e ultimo vertice serao 'src'. Se for um P3, um possiveo retorno seria [0, 1, 2, 0] get_cycle() acha um ciclo euleriano se o grafo for euleriano. Se for um P3, um possivel retorno seria [0, 1, 2] (vertie inicial nao repete) O(n+m)*

```
f83 template<bool directed=false> struct euler {
5c1     ll n;
d2a     vector<vector<p64>> g;
ae8     v64 used;

88a     euler(ll n_) : n(n_), g(n) {}
1a7     void add(ll a, ll b) {
f55         ll at = used.size();
19c         used.push_back(0);
205         g[a].emplace_back(b, at);
072         if (!directed) g[b].emplace_back(a, at);
2f3     }
8d4 #warning chamar para o src certo!
71d     pair<bool, vector<p64>> get_path(ll src) {
540         if (!used.size()) return {true, {}};
80b         v64 beg(n, 0);
f4b         for (ll& i : used) i = 0;
         // {{vertice, anterior}, label}
3ee         vector<pair<p64, ll>> ret, st = {{{src, -1}, -1}};
90c         while (st.size()) {
4fb             ll at = st.back().first.first;
6ed             ll& it = beg[at];
5b4             while (it < g[at].size() and used[g[at][it].second])
 it++;
d6d             if (it == g[at].size()) {
58e
                if (ret.size() and ret.back().first.second != at)
f00                     return {false, {}};
cb5                 ret.push_back(st.back()), st.pop_back();
2cd             } else {
```

```
bb0                 st.push_back({{g[at][it].first, at}, g[at][it].
second});
6b3                 used[g[at][it].second] = 1;
610             }
6ca         }
e6f         if (ret.size() != used.size()+1) return {false, {}};
6fe         vector<p64> ans;
94c         for (auto i : ret) ans.emplace_back(i.first.first, i.
second);
d4f         reverse(ans.begin(), ans.end());
d78         return {true, ans};
fde     }
213     pair<bool, vector<p64>> get_cycle() {
540         if (!used.size()) return {true, {}};
934         ll src = 0;
021         while (!g[src].size()) src++;
f3e         auto ans = get_path(src);
ecc         if (!ans.first or ans.second[0].first != ans.second.back
().first)
f00             return {false, {}};
9bd         ans.second[0].second = ans.second.back().second;
806         ans.second.pop_back();
ada         return ans;
cc2     }
2f4 };
```

## 2.11 Floyd-Warshall Algorithm 340

*Computes all-pairs shortest paths via DP over intermediate vertices. Initialize d[i][j] = edge weight (INF if no edge, 0 on diagonal). Returns true if a negative cycle is detected.*

**Complexity:** O(N{}3), O(N{}2)

```
aa7 bool floyd_warshall(ll n, vector<v64>& d) {
6a4     forn(k,0,n)
33f     forn(i,0,n)
9e3     forn(j,0,n)
c74         if (d[i][k] != INF && d[k][j] != INF)
3c1             d[i][j] = min(d[i][j], d[i][k] + d[k][j]);

33f     forn(i,0,n)
b4d         if (d[i][i] < 0) return true;

b09     return false;
340 }
```

## 2.12 Heavy Light Decomposition (Path query + Path update) 378

*Decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most log(n) light edges. All values initialized to the segtree default. Root must be 0. Uses a Lazy Segment Tree*

**Complexity:** O((log N){}2) per op, O(N) build

```
e8f #include "src/structures/segtree_lazy.cpp"

273 template <bool VALS_EDGES> struct HLD {
657     ll N, tim = 0;
02a     vector<v64> adj;
f53     v64 parent, siz, head, pos;
b32     vector<Node> vseg;
e73     std::unique_ptr<segtree<Node, Update>> seg;
ae4     HLD(vector<v64> adj_, v64 vals)
7c1         : N(sz(adj_)), adj(adj_), parent(N, -1), siz(N, 1),
1dd           head(N),pos(N),vseg(N, {0}){ dfsSz(0); dfsHld(0);
aec             seg = make_unique<segtree<Node, Update>>(N);
d8f             seg->set_leaves(vseg);
0b3         }
d5e     void dfsSz(ll v) { // get heavy son
d17         for (ll& u : adj[v]) {
ac6             adj[u].erase(find(adj[u].begin(), adj[u].end(), v));
762             parent[u] = v;
336             dfsSz(u);
ec7             siz[v] += siz[u];
1e2             if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
561         }
bc8     }
016     void dfsHld(ll v) { // "linearizes" the tree
9e4         pos[v] = tim++;
ca5         for (ll u : adj[v]) {
63f             head[u] = (u == adj[v][0] ? head[v] : u);
8b3             dfsHld(u);
8ff         }
c03     }
ed1     template <class B> void process(ll u, ll v, B op) {
aea         for (;; v = parent[head[v]]) {
c30             if (pos[head[u]] > pos[head[v]]) swap(u, v);
152             if (head[u] == head[v]) break;
983             op(pos[head[v]], pos[v]);
82b         }
770         if (pos[u] > pos[v]) swap(u, v);
fd8         op(pos[u] + VALS_EDGES, pos[v]);
00e     }
546     void modifyPath(ll u, ll v, ll val) {
491         process(u, v, [&](ll l, ll r) {
65b
            seg->update(l, r, {val}); // Modify depending on problem
516         });
8b0     }
770     ll queryPath(ll u, ll v) { // Modify depending on problem
b3a         ll res = -INF;
491         process(u, v, [&](ll l, ll r) {
878                 res = max(res, seg->query(l, r).val);
5bc         });
8df         return res;
16d     }
20d     ll querySubtree(ll v) { // modifySubtree is similar
e8c         return seg->query(pos[v] + VALS_EDGES, pos[v] + siz[v] -
 1).val;
130     }
260 };
```

## 2.13 Hungarian algorithm — min-cost perfect matching ac8

*a[i][j] = cost of assigning worker i to job j (n workers, m >= n jobs) Returns {min cost, assignment}: assignment[i] = j*

```
89a pair<ll, v64> hungarian(const vector<v64> &a) {
c3e     if (a.empty()) return {0, {}};
708     ll n = sz(a) + 1, m = sz(a[0]) + 1;
fb8     v64 u(n), v(m), p(m), ans(n - 1);
6ad     forn(i, 1, n) {
2cb         p[0] = i;
c00         ll j0 = 0; // "dummy" worker 0
b7f         v64 dist(m, INF), pre(m, -1);
3ce         vector<bool> done(m + 1);
f88         do { // dijkstra
91a             done[j0] = true;
86d             ll i0 = p[j0], j1 = 0, delta = INF;
7cf             forn(j, 1, m) if (!done[j]) {
704                 auto cur = a[i0 - 1][j - 1] - u[i0] - v[j];
fb6                 if (cur < dist[j]) dist[j] = cur, pre[j] = j0;
5c3                 if (dist[j] < delta) delta = dist[j], j1 = j;
e05             }
58f             forn(j, 0, m) {
0c5                 if (done[j]) u[p[j]] += delta, v[j] -= delta;
c6f                 else dist[j] -= delta;
405             }
afc             j0 = j1;
f1a         } while (p[j0]);
e10         while (j0) { // update alternating path
d08             ll j1 = pre[j0];
6ae             p[j0] = p[j1], j0 = j1;
013         }
dbc     }
973     forn(j, 1, m) if (p[j]) ans[p[j] - 1] = j - 1;
eb4     return {-v[0], ans}; // min cost
ac8 }
```

## 2.14 LCA com Euler tour + Sparse Table (RMQ) ab9

*Constroi com o grafo (lista de adjacencia) e a raiz 0 Assume arvore enraizada em 0 Complexidades: build - O(n log(n)) lca - O(1) dist - O(1)*

```
4b6 #include "src/structures/sparse_table.cpp"

333 struct LCA {
fc7     ll T = 0;
4c3     v64 time, path, ret, dep;
632     sparse rmq;

757     LCA(vector<v64>& C, ll root) : time(sz(C)), dep(sz(C)), rmq
((dfs(C,root,-1), ret)) {}
181     void dfs(vector<v64>& C, ll v, ll par) {
cc2         time[v] = T++;
33a         if(par != -1) dep[v] = dep[par] + 1;
5f8         for (ll y : C[v]) if (y != par) {
f50             path.push_back(v), ret.push_back(time[v]);
f70             dfs(C, y, v);
36d         }
518     }

002     ll lca(ll a, ll b) {
bb6         if (a == b) return a;
bd3         tie(a, b) = minmax(time[a], time[b]);
c99         return path[rmq.query(a, b-1)];
558     }
b5d
    ll dist(ll a,ll b){return dep[a] + dep[b] - 2*dep[lca(a,b)];}
880 };
```

## 2.15 LCA com binary lifting 7de

*Assume que um vertice eh ancestral dele mesmo, ou seja, se a eh ancestral de b, lca(a, b) = a MAXLOG = ceil(log(MAX)) Complexidades: build - O(n log(n)) lca - O(log(n))*

```
202 vector<v64> g(MAX);
7e0 ll n, p;
9e8 ll parent[MAXLOG][MAX];
c5e ll in[MAX], out[MAX];

fd3 void dfs(ll k) {
18f     in[k] = p++;
654     for(ll v: g[k]){
```

```
050         if(in[v] == -1){
e10             parent[0][v] = k;
a7e             dfs(v);
a67         }
3a5     }
c34     out[k] = p++;
ab4 }

bf8 void build(ll raiz) {
46c     forn(i,0,n) parent[0][i] = i;
014     p = 0, memset(in, -1, sizeof in);
6b5     dfs(raiz);
104     forn(k,1,MAXLOG) forn(i,0,n)
423         parent[k][i] = parent[k - 1][parent[k - 1][i]];
896 }

fb3 bool anc(ll a, ll b) { // se a eh ancestral de b
b01     return in[a] <= in[b] && out[a] >= out[b];
306 }

002 ll lca(ll a, ll b) {
8ff     if (anc(a, b)) return a;
828     if (anc(b, a)) return b;
084     for (ll k = MAXLOG - 1; k >= 0; k--)
5f1         if (!anc(parent[k][a], b)) a = parent[k][a];
6a3     return parent[0][a];
b4f }
```

## 2.16 Min Cost Max Flow (MCMF) 10d

*min_cost_flow(s, t, f) computes the pair (flow, cost) where max(flow) <= f with min(cost). min_cost_flow(s, t) -> Maximum flow with minimum cost from s to t. For DAGs, SPFA can be replaced with DP to avoid O(nm) initial cost. If no negative cost edges exist, SPFA is not needed.*

**Complexity:** O(nm + f * m log n), O(n + m)

```
4ad template<typename T> struct mcmf {
60e     struct edge {
500         ll to, rev, flow, cap; // destination, reverse edge id, current flow, capacity
1c9         bool res; // is reverse edge
ccb         T cost; // cost per unit of flow
6d3         edge() : to(0), rev(0), flow(0), cap(0), cost(0), res(false) {}
6db         edge(ll to_, ll rev_, ll flow_, ll cap_, T cost_, bool res_)
883             : to(to_), rev(rev_), flow(flow_), cap(cap_), res(res_), cost(cost_) {}
8e9     };

b0e     vector<vector<edge>> g;
c97     v64 par_idx, par;
6f5     T inf;
cdb     vector<T> dist;

9aa     mcmf(ll n) : g(n), par_idx(n), par(n), inf(numeric_limits<T>::max()/3) {}

45f     void add(ll u, ll v, ll w, T cost) { // edge from u to v with capacity w and cost
b6f         edge a = edge(v, sz(g[v]), 0, w, cost, false);
6db         edge b = edge(u, sz(g[u]), 0, 0, -cost, true);

4c5         g[u].push_back(a);
6c1         g[v].push_back(b);
72a     }

5cd     vector<T> spfa(ll s) { // not needed if no negative cost edges
6d1         deque<ll> q;
f30         vector<bool> is_inside(sz(g), 0);
f73         dist = vector<T>(sz(g), inf);

81e         dist[s] = 0;
261         q.push_back(s);
ded         is_inside[s] = true;

b27         while (!q.empty()) {
bf4             ll v = q.front();
be6             q.pop_front();
f1e             is_inside[v] = false;

b24             forn(i,0,sz(g[v])){
7ef                 auto [to, rev, flow, cap, res, cost] = g[v][i];
446                 if (flow < cap && dist[v] + cost < dist[to]) {
60c                     dist[to] = dist[v] + cost;

166                     if (is_inside[to]) continue;
ff5                     if (!q.empty() && dist[to] > dist[q.front()]) q.push_back(to);
7d3                     else q.push_front(to);
4ed                     is_inside[to] = true;
ceb                 }
d67             }
ef5         }
c9d         return dist;
48a     }
0e9     bool dijkstra(ll s, ll t, vector<T>& pot) {
c2f         priority_queue<pair<T, ll>, vector<pair<T, ll>>, greater<>> q;
f73         dist = vector<T>(sz(g), inf);
81e         dist[s] = 0;
aa9         q.emplace(0, s);
dad         while (sz(q)) {
7df             auto [d, v] = q.top();
f93             q.pop();
e90             if (dist[v] < d) continue;
b24             forn(i,0,sz(g[v])) {
7ef                 auto [to, rev, flow, cap, res, cost] = g[v][i];
3c0                 cost += pot[v] - pot[to];
446                 if (flow < cap && dist[v] + cost < dist[to]) {
60c                     dist[to] = dist[v] + cost;
f90                     q.emplace(dist[to], to);
921                     par_idx[to] = i, par[to] = v;
633                 }
fb9             }
af9         }
98b         return dist[t] < inf;
b13     }

8c4     pair<ll, T> min_cost_flow(ll s, ll t, ll flow = INF) {
947         vector<T> pot(sz(g), 0);
da6         pot = spfa(s); // change shortest path algorithm here
            // no negative costs: nothing needed
            // DAG: use DP instead
d3a         ll f = 0;
8a4         T ret = 0;
8cf         while (f < flow && dijkstra(s, t, pot)) {
afe             forn(i,0,sz(g))
85f                 if (dist[i] < inf) pot[i] += dist[i];

dda             ll mn_flow = flow - f, u = t;
c1a             while (u != s){
715                 mn_flow = min(mn_flow,
582                     g[par[u]][par_idx[u]].cap - g[par[u]][par_idx[u]].flow);
a09                 u = par[u];
27c             }

50a             ret += pot[t] * mn_flow;

060             u = t;
c1a             while (u != s) {
5b9                 g[par[u]][par_idx[u]].flow += mn_flow;
e52                 g[u][g[par[u]][par_idx[u]].rev].flow -= mn_flow;
a09                 u = par[u];
47d             }

335             f += mn_flow;
350         }

b5a         return make_pair(f, ret);
e41     }

    // Optional: returns original edges where flow == cap
5d9     vector<p64> recover() {
0b1         vector<p64> used;
7c2         forn(i, 0, sz(g)) for (edge e : g[i])
e76             if(e.flow == e.cap && !e.res) used.push_back({i, e.to});
992         return used;
997     }
10d };
```

## 2.17 Strongly Connected Components (Kosaraju) af8

*Computes SCCs using two DFS passes and builds the condensation graph.*

**Complexity:** O(N + M), O(N + M)

```
ed6 vector<bool> visited;

be2 void dfs(ll v, vector<v64>& g, vector<ll> &out) {
9dc     visited[v] = true;
619     for(auto u : g[v]) if(!visited[u]) dfs(u, g, out);
8a8     out.push_back(v);
65c }

c44 vector<v64> scc(vector<v64>& g) {
92e     int n = sz(g);
10e     v64 order, roots(n, 0);

ad4     vector<v64> adj_rev(n);
0d6     forn(u, 0, n) for (ll v : g[u]) adj_rev[v].push_back(u);

e18     visited.assign(n, false);
7bf     forn(i, 0, n) if (!visited[i]) dfs(i, g, order);
962     reverse(order.begin(), order.end());

e18     visited.assign(n, false);
a13     ll curr_comp = 0;
1a4     for (auto v : order) {
```

```
936         if (!visited[v]) {
304             v64 component; dfs(v, adj_rev, component);
11f             for (auto u : component) roots[u] = curr_comp;
b12             curr_comp++;
b46         }
bed     }

697     set<p64> edges;
5e8     vector<v64> cond_g(curr_comp);
4c4     forn(u, 0, n) {
45c         for (auto v : g[u]) {
a88             if (roots[u] != roots[v] && !edges.count({roots[u],
roots[v]})) {
bbd                 cond_g[roots[u]].push_back(roots[v]);
cf1                 edges.emplace(roots[u], roots[v]);
ee1             }
e33         }
9b3     }
325     return cond_g;
893 }
```

### 2.18 Topological Sort (Kahn's Algorithm) 9b8

*Produces a topological ordering of a DAG using indegree counting and a queue-like frontier.*

**Complexity:** O(N + M), O(N)

```
a08 v64 topo_sort(const vector<v64>& g) {
d0d     v64 indeg(sz(g)), q;
635     for (auto& li : g) for (int x : li) indeg[x]++;
8da     forn(i, 0, sz(g)) if (indeg[i] == 0) q.push_back(i);
d61     forn(j, 0, sz(q)) for(int x : g[q[j]]) if(--indeg[x] == 0) q.
push_back(x);
2e8     return q;
9b8 }
```

### 2.19 Virtual Tree 1f7

*Comprime uma arvore dado um conjunto S de vertices, de forma que o conjunto de vertices da arvore comprimida contenha S e seja minimal e fechado sobre a operacao de LCA Se |S| = k, a arvore comprimida tem menos que 2k vertices As arestas de virt possuem a distancia do vertice ate o vizinho em arestas Retorna a raiz da virtual tree Precisa: lca::pos deve ser a ordem de visitacao no dfs lca::lca, lca::dist O(k * T), T tempo do lca::lca*

```
aa4 #include "src/graphs/lca.cpp"

c17 const ll MAX = 100'005;
528 vector<pair<ll, ll>> virt[MAX];

a9e ll build_virt(vector<ll> v, LCA& lca) {
538     auto cmp = [&](ll i, ll j) { return lca.time[i] < lca.time[j
]; };
da4     sort(v.begin(), v.end(), cmp);
0c7
    for (ll i = sz(v)-1; i; i--) v.push_back(lca.lca(v[i], v[i-1]));
da4     sort(v.begin(), v.end(), cmp);
cd9     v.erase(unique(v.begin(), v.end()), v.end());
b84     forn(i,0,sz(v)) virt[v[i]].clear();
4cd     forn(i,1,sz(v)) virt[lca.lca(v[i-1], v[i])].clear();
17a     forn(i,1,sz(v)) {
270         ll parent = lca.lca(v[i-1], v[i]);
c88         ll d = lca.dist(parent, v[i]);
 // #warning soh to colocando aresta descendo
f33         virt[parent].emplace_back(v[i], d);
d7a     }

ec5     return v[0];
a33 }
```

# 3 Mathematics

### 3.1 Chinese Remainder Theorem 603

*se x = a(mod m) e x = b(mod n) então existe unico c<mmc(m,n) tal que x = c(mod mmc(m,n)) se e só se mdc(m,n)| (b-a) essa funcao acha c e é -INF se for impossivel*

```
b12 #include "src/math/euclid.cpp"

4cf ll crt(ll a, ll m, ll b, ll n) {
15c     if (n > m) swap(a, b), swap(m, n);
85f     ll x, y, g = euclid(m, n, x, y);
12c     if((a - b) % g != 0) return -INF; //no solution
467     x = (b - a) % n * x % n / g * m + a;
187     return x < 0 ? x + m*n/g : x;
c44 }
```

### 3.2 Extended Eucidian Algorithm 1b3

*Acha mdc(a,b) e x e y que satisfazem a identidade de bezout*

```
429 ll euclid(ll a, ll b, ll &x, ll &y) {
b89     if (!b) return x = 1, y = 0, a;
1df     ll d = euclid(b, a % b, y, x);
f58     return y -= a/b * x, d;
1b3 }
```

### 3.3 FFT/NTT Convolution d95

*Implements iterative FFT over complex numbers and NTT over supported primes; provides convolution utility.*

**Complexity:** O(N log N), O(N)

```
8ac void get_roots(bool f, int n, vector<complex<double>>& roots) {
679     const static double PI = acosl(-1);
96d     for (int i = 0; i < n/2; i++) {
8e4         double alpha = i*((2*PI)/n);
233         if (f) alpha = -alpha;
4b4         roots[i] = {cos(alpha), sin(alpha)};
1e6     }
e85 }

// Para NTT
65a template<int p>
8ab void get_roots(bool f, int n, vector<mod_int<p>>& roots) {
47f     mod_int<p> r;
df5     int ord;
2c5     if (p == 998244353) {
620         r = 102292;
350         ord = (1 << 23);
a9f     } else if (p == 754974721) {
530         r = 739831874;
00d         ord = (1 << 24);
f8d     } else if (p == 167772161) {
13a         r = 243;
84e         ord = (1 << 25);
3c2     } else assert(false);

e6b     if (f) r = r^(p - 1 -ord/n);
1a7     else r = r^(ord/n);
dcd     roots[0] = 1;
5ea     for (int i = 1; i < n/2; i++) roots[i] = roots[i-1]*r;
892 }

27e template<typename T> void fft(vector<T>& a, bool f, int N,
vector<int>& rev) {
179     for (int i = 0; i < N; i++) if (i < rev[i]) swap(a[i], a[rev
[i]]);
4a1     int l, r, m;
a38     vector<T> roots(N);
66d     for (int n = 2; n <= N; n *= 2) {
13c         get_roots(f, n, roots);
017         for (int pos = 0; pos < N; pos += n) {
235             l = pos + 0, r = pos + n/2, m = 0;
0a7             while (m < n/2) {
451                 auto t = roots[m] * a[r];
54c                 a[r] = a[l] - t;
db7                 a[l] = a[l] + t;
c26                 l++, r++, m++;
a3a             }
db8         }
925     }
80d     if (f) {
dd9         auto invN = T(1) / T(N);
667         for (int i = 0; i < N; i++) a[i] = a[i] * invN;
d42     }
a5b }

9f6 template<typename T> vector<T> convolution(vector<T>& a, vector<
T>& b) {
902     vector<T> l(a.begin(), a.end()), r(b.begin(), b.end());
927     int N = sz(l)+sz(r)-1;
aea     int n = 1, log_n = 0;
884     while (n <= N) n *= 2, log_n++;
dea     vector<int> rev(n);
8ee     for (int i = 0; i < n; i++) {
23e         rev[i] = 0;
f95         for (int j = 0; j < log_n; j++) if (i>>j&1)
2be             rev[i] |= 1 << (log_n-1-j);
046     }

2f0     assert(N <= n);
5c1     l.resize(n);
ca3     r.resize(n);
769     fft(l, false, n, rev);
4eb     fft(r, false, n, rev);
186     for (int i = 0; i < n; i++) l[i] *= r[i];
99d     fft(l, true, n, rev);
758     l.resize(N);
273     return l;
55a }

// NTT
b09 template<int p, typename T>
ebc vector<mod_int<p>> ntt(vector<T>& a, vector<T>& b) {
3b0
    vector<mod_int<p>> A(a.begin(), a.end()), B(b.begin(), b.end());
697     return convolution(A, B);
32b }
```

### 3.4 Divisor Decomposition / Square Root Trick c38

*Itera sobre intervalos [l, r] onde floor(n/i) é constante Complexidade: O(sqrt(n))*

```
493 void divisor_decomposition(ll n) {
```

```
1d8     for (ll l = 1, r; l <= n; l = r + 1) {
4ae         ll val = n / l;
d4a         r = n / val;
         // n/i == val para todo i em [l, r]
         // Exemplo: res += (r - l + 1) * val;
21f     }
c38 }
```

## 3.5 Gauss 7a5

*Resolve sistema linear Retornar um par com o numero de solucoes e alguma solucao, caso exista O(n{}2 * m) escrito pelo gpt, cuidado*

```
442 template<typename T>
0b9 pair<ll, vector<T>> gauss(vector<vector<T>> a, vector<T> b) {
ef1     const double eps = 1e-6;
337     ll n = sz(a), m = sz(a[0]);
fcb     forn(i, 0, n) a[i].push_back(b[i]);

af0     v64 where(m, -1);
afc     for (ll col = 0, row = 0; col < m && row < n; col++) {
caa         ll sel = row;
ed1         forn(i, row, n)
4c9             if(abs(a[i][col]) > abs(a[sel][col])) sel = i;
912         if(abs(a[sel][col]) < eps) continue;
63f         forn(i, col, m+1)
554             swap(a[sel][i], a[row][i]);
a32         where[col] = row;

c62         forn(i, 0, n) if (i != row) {
5f0             T c = a[i][col] / a[row][col];
36d             forn(j, col, m+1)
c18                 a[i][j] -= a[row][j] * c;
e08         }
42f         row++;
2fd     }

88e     vector<T> ans(m, 0);
b49     forn(i, 0, m) if (where[i] != -1)
825         ans[i] = a[where[i]][m]/a[where[i]][i];
bda     forn(i, 0, n) {
655         T sum = 0;
96b         forn(j, 0, m)
345             sum += ans[j]*a[i][j];
d72         if(abs(sum - a[i][m]) > eps)
63a             return pair(0, vector<T>());
db1     }

445     forn(i, 0, m) if (where[i] == -1)
335         return pair(INF, ans);
abd     return pair(1, ans);
15a }
```

## 3.6 Gauss - Z2 2b9

*D eh dimensao do espaco vetorial add(v) - adiciona o vetor v na base (retorna se ele jah pertencia ao span da base) coord(v) - retorna as coordenadas (c) de v na base atual (basis{}T.c = v) recover(v) - retorna as coordenadas de v nos vetores na ordem em que foram inseridos coord(v).first e recover(v).first - se v pertence ao span Complexidade: add, coord, recover: O(D{}2 / 64)*

```
4ee template<ll D> struct gauss_z2 {
5f5     bitset<D> basis[D], keep[D];
3f8     ll rk, in, dim;
1f7     v64 id;

b3e     gauss_z2 () : rk(0), in(-1), id(D, -1) {};

fb2     gauss_z2 (ll dim_) : rk(0), in(-1), id(D, -1), dim(dim_) {};

25a     bool add(bitset<D> v) {
4ef         in++;
595         bitset<D> k;
d1a         for (ll i = dim - 1; i >= 0; i--) if (v[i]) {
e39             if (basis[i][i]) v ^= basis[i], k ^= keep[i];
12a             else {
583                 k[i] = true, id[i] = in, keep[i] = k;
5ab                 basis[i] = v, rk++;
01d                 return true;
5e6             }
16e         }
b09         return false;
acb     }
e7f     pair<bool, bitset<D>> coord(bitset<D> v) { // private
314         bitset<D> c;
d1a         for (ll i = dim - 1; i >= 0; i--) if (v[i]) {
a26             if (basis[i][i]) v ^= basis[i], c[i] = true;
17c             else return {false, bitset<D>()};
ef0         }
cef         return {true, c};
608     }
2c7     pair<bool, v64> recover(bitset<D> v) {
030         auto [span, bc] = coord(v);
319         if (not span) return {false, {}};
c28         bitset<D> aux;
ebc
        for (ll i = dim - 1; i >= 0; i--) if (bc[i]) aux ^= keep[i];
a30         v64 oc;
e11         for (ll i = dim - 1; i >= 0; i--) if (aux[i]) oc.
push_back(id[i]);
371         return {true, oc};
78d     }
2b9 };
```

## 3.7 Flattened Divisor Sieve (CSR format) f39

*Precomputes all divisors up to n. Faster than vector<v64> for cache reasons Time Complexity: O(n log n) | Space Complexity: O(n log n)*

```
a0f tuple<v64, v64, v64> get_divisor(ll n) {
851     v64 head(n);
136     v64 tail(n);
8bd     v64 divs(n*(log(n)+1));

862     forn(i, 1, n)
56d         for (ll j = i; j < n; j += i)
5b3             head[j]++;

1ef     ll curr = 0;
6ad     forn(i, 1, n) {
982         ll cnt = head[i];
50c         head[i] = curr;
bd9         tail[i] = curr;
084         curr += cnt;
e48     }

862     forn(i, 1, n)
56d         for (ll j = i; j < n; j += i)
f5e             divs[tail[j]++] = i;

0ab     return {head, tail, divs};
f39 }
// for d | x =
 // forn(k,head[x], tail[x]){
 //     ll d = divs[k];
// }
```

## 3.8 Matrix template 720

*- operator*: multiplicacao (O(n*m*r)) opcional modo modular (#define MODULAR true) - operator{}: exponenciacao rapida (O(log e)) - apply_transform(): aplica transformacao M{}e * v*

```
205 #define MODULAR false
692 template<typename T> struct matrix : vector<vector<T>> {
908     ll n, m;
f23     void print() {
bda         forn(i,0,n) {
80b             forn(j,0,m) cout << (*this)[i][j] << " ";
8a6             cout << endl;
aeb         }
40e     }

e9c     matrix(ll n_, ll m_, bool ident = false) :
b03
            vector<vector<T>>(n_, vector<T>(m_, 0)), n(n_), m(m_) {
1ed         if (ident) {
e7b             assert(n == m);
e7f             forn(i,0,n) (*this)[i][i] = 1;
122         }
a86     }
29c     matrix(const vector<vector<T>>& c) : vector<vector<T>>(c),
133         n(sz(c)), m(sz(c[0])) {}
1b5     matrix(const initializer_list<initializer_list<T>>& c) {
7dc         vector<vector<T>> val;
de7         for (auto& i : c) val.push_back(i);
cec         *this = matrix(val);
f06     }

f86     matrix<T> operator*(matrix<T>& r) {
43c         assert(m == r.n);
c2e         matrix<T> M(n, r.m);
966         forn(i,0,n) forn(k,0,m) forn(j, 0, r.m){
bc4             T add = (*this)[i][k] * r[k][j];
f98 #if MODULAR
8d4 #warning Usar matrix<ll> e soh colocar valores em [0, MOD) na
matriz!
ba1             M[i][j] += add%MOD;
795             if (M[i][j] >= MOD) M[i][j] -= MOD;
8c1 #else
3f3             M[i][j] += add;
f2e #endif
5bb         }
3f1         return M;
2de     }
2e3     matrix<T> operator^(ll e){
6ff         matrix<T> M(n, n, true), at = *this;
382         while (e) {
92d             if (e&1) M = M*at;
133             e >>= 1;
5ef             at = at*at;
b1a         }
3f1         return M;
84d     }
```

```
f49     void apply_transform(matrix M, ll e){
dd9         auto& v = *this;
382         while (e) {
394             if (e&1) v = M*v;
133             e >>= 1;
cd1             M = M*M;
01f         }
660     }
955 };
```

### 3.9 Miller-Rabin Primality Test (Deterministic for 64-bit) bdb

*Checks if a number N is prime using a fixed set of bases that guarantees correctness for any N < 2{}64. Uses a long double trick for O(1) modular multiplication.*

**Complexity:** O(7 log{}3 N) per query

```
924 #include "src/math/mod_functions.cpp"

cfd bool isPrime(ull n){
66c     if(n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
b13     ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
 s = __builtin_ctzll(n-1), d = n >> s;
cd4     for (ull a : A) {
471         ull p = modpow(a%n, d, n), i = s;
0c1         while(p != 1 && p != n-1 && a % n && i--)
b4c             p = modmul(p, p, n);
a6f         if(p != n-1 && i != s) return 0;
815     }
af7     return 1;
518 }
```

### 3.10 Modular Integer 123

*Fixed-modulus integer type with +, -, *, /, and exponentiation; modulo should be prime for division via Fermat.*

**Complexity:** O(1) per arithmetic op (O(log E) for exponentiation), O(1)

```
335 const ll MOD = 998244353;

7a5 template<ll p> struct mod_int {
d27     ll expo(ll b, ll e) {
2b3         ll ret = 1;
382         while (e) {
9dc             if (e % 2) ret = ret * b % p;
f84             e /= 2, b = b * b % p;
30f         }
3cb         return ret;
931     }
7c0     ll inv(ll b) { return expo(b, p-2); }

591     using m = mod_int;
da9     ll v;
68b     mod_int() : v(0) {}
89c     mod_int(ll v_) {
323         if (v_ >= p || v_ <= -p) v_ %= p;
95b         if (v_ < 0) v_ += p;
6b4         v = v_;
3ac     }
e8a     m& operator +=(const m& a) {
385         v += a.v;
2f4         if (v >= p) v -= p;
ace         return *this;
2e9     }
681     m& operator -=(const m& a) {
cc9         v -= a.v;
1db         if (v < 0) v += p;
ace         return *this;
b87     }
f5d     m& operator *=(const m& a) {
717         v = v * a.v % p;
ace         return *this;
5d9     }
5ae     m& operator /=(const m& a) {
5b9         v = v * inv(a.v) % p;
ace         return *this;
b8b     }
1fd     m operator -() const { return m(-v); }
188     m& operator ^=(ll e) {
a64         if (e < 0) {
639             v = inv(v);
1e0             e = -e;
056         }
3b5         v = expo(v, e);
         // possible optimization:
         // careful with 0^0
         // v = expo(v, e%(p-1));
ace         return *this;
17e     }
c01     bool operator ==(const m& a) { return v == a.v; }
9b6     bool operator !=(const m& a) { return v != a.v; }

876     friend istream& operator >>(istream& in, m& a) {
13b         ll val; in >> val;
9e8         a = m(val);
701         return in;
6c3     }
9fe     friend ostream& operator <<(ostream& out, m a) {
8a9         return out << a.v;
55b     }
bc1     friend m operator +(m a, m b) { return a += b; }
656     friend m operator -(m a, m b) { return a -= b; }
fd6     friend m operator *(m a, m b) { return a *= b; }
d6f     friend m operator /(m a, m b) { return a /= b; }
c67     friend m operator ^(m a, ll e) { return a ^= e; }
c07 };
47b typedef mod_int<MOD> mint;
```

### 3.11 Modular Arithmetic Helpers f78

*Provides fast modular mul, fast exponentiation.*

**Complexity:** O(log E) for power/inverse

```
f9c typedef unsigned long long ull;
55e const ll MOD = 1'000'000'007;

12d ull modmul(ull a, ull b, ull M){
279     ll ret = a*b - M * ull(1.L / M*a*b);
e7f     return ret + M * (ret < 0) - M * (ret >= (ll)M);
b54 };

a73 ull modpow(ull b, ull e, ull mod){
d17     ull ans = 1;
91a     for(; e; b = modmul(b, b, mod), e /= 2)
755         if(e & 1) ans = modmul(ans, b, mod);
ada     return ans;
ec4 }
```

### 3.12 Discrete Logarithm (Baby-step Giant-step) 9f9

*Finds the smallest x >= 1 such that a{}x = b (mod m), or returns -1 if no solution exists. Uses Baby-step Giant-step with a hash map for O(sqrt(m)) lookups.*

**Complexity:** O(sqrt(m) * log(sqrt(m)))

```
3af ll mod_log(ll a, ll b, ll m) {
17d     ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1;
6f3     map<ll, ll> mp;
50c     while (j <= n && (e = f = e * a % m) != b % m)
8c2         mp[e * b % m] = j++;
7a1     if (e == b % m) return j;
9f7     if (gcd(m, e) == gcd(m, b))
3fc         forn(i,2,n+2) if (mp.count(e = e * f % m))
48f             return n * i - mp[e];
68e     return -1;
9f9 }
```

### 3.13 Discrete Square Root ef7

*Tonelli-Shanks para encontrar x tq x{}2 = a (mod p) O(sqrt(p))*

```
924 #include "src/math/mod_functions.cpp"

cc6 ll sqrt(ll a, ll p) {
876     a %= p; if (a < 0) a += p;
9d6     if (a == 0) return 0;
c63     if(modpow(a, (p-1)/2, p) != 1) return -1; // se não é
possivel fala -1
87c     if (p % 4 == 3) return modpow(a, (p+1)/4, p);
     // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
f38     ll s = p - 1, n = 2;
645     int r = 0, m;
2e2     while (s % 2 == 0)
343         ++r, s /= 2;
     /// find a non-square mod p
db5     while (modpow(n, (p - 1) / 2, p) != p - 1) ++n;
a9d     ll x = modpow(a, (s + 1) / 2, p);
91e     ll b = modpow(a, s, p), g = modpow(n, s, p);
ed1     for (;; r = m) {
ac6         ll t = b;
75b         for (m = 0; m < r && t != 1; ++m)
e70             t = t * t % p;
efd         if (m == 0) return x;
d27         ll gs = modpow(g, 1LL << (r - m - 1), p);
f7d         g = gs * gs % p;
996         x = x * gs % p;
e2e         b = b * g % p;
a1f     }
738 }
```

### 3.14 Primitive Root (Generator) f3f

*Finds the smallest primitive root modulo a prime P. Factorizes P-1 and checks that for every prime factor F of P-1, the candidate G satisfies G{}((P-1)/F) != 1 (mod P). can be adapted to p not prime*

**Complexity:** O(time_to_factor(p-1) + ans * |factors(P-1)| * log P)

```
924 #include "src/math/mod_functions.cpp"

091 ll primitive_root(ll p){
6ab     v64 fact;
16a     ll phi = p-1,  n = phi;
     // mudar p-1 para caso nao primo
4e1     for (ll i=2; i*i<=n; i++)
```

```
ed3         if (n % i == 0) {
e0f             fact.push_back (i);
b71             while (n % i == 0)
a3e                 n /= i;
294         }
dc8     if (n > 1)
7e2         fact.push_back(n);

8ce     forn(res,2, p+1) {
e46         bool ok = true;
893         for (ll i=0; i<sz(fact) && ok; ++i)
7c9             ok &= modpow (res, phi / fact[i], p) != 1;
c34         if (ok)  return res;
10a     }
68e     return -1;
67b }
```

### 3.15 Quadratic Field Extension (Z_p[sqrt(D)]) b77

*Extension for numbers of form x + y*sqrt(D) with +, -, *, /, and exponentiation.*
**Complexity:** O(1) per arithmetic op, O(log MOD) for division/inversion.

```
360 #include "src/math/mint.cpp"

6cc template <long long D>
310 struct quad {
4ed     mint x, y;
787     using m = quad<D>;

300     quad() : x(0), y(0) {}
2c2     quad(ll x_) : x(x_), y(0) {}
a70     quad(mint x_, mint y_) : x(x_), y(y_) {}

e8a     m& operator += (const m& a) {
03c         x += a.x; y += a.y; return *this;
31f     }
681     m& operator -= (const m& a) {
29a         x -= a.x; y -= a.y; return *this;
e14     }

f5d     m& operator *= (const m& a) {
dd4         mint old_x = x;
0a7         x = x * a.x + mint(D) * y * a.y;
52d         y = old_x * a.y + y * a.x;
ace         return *this;
d3b     }

473     m inv() const {
738         mint dem = x * x - mint(D) * y * y;
d57         return m(x / dem, -y / dem);
aca     }

5ae     m& operator /=(const m& a) {
feb         *this *= a.inv();
ace         return *this;
e18     }

1ca     m expo(m b, ll e){
f50         m ret = m(1, 0);
382         while (e) {
700             if (e % 2) ret = ret * b;
499             b = b * b;
9a2             e /= 2;
dfd         }
3cb         return ret;
0ca     }

188     m& operator ^=(ll e) {
a64         if (e < 0) {
9b6             *this = this->inv();
1e0             e = -e;
f7e         }
315         *this = expo(*this, e);
ace         return *this;
78c     }

8f5
    bool operator ==(const m& a){ return (x == a.x) && (y == a.y); }
0c5
    bool operator !=(const m& a){ return (x != a.x) || (y != a.y); }

dbe     friend ostream& operator <<(ostream& out, const m& a) {
88c         return out << a.x << ":" << a.y;
fed     }

382     friend m operator +(m a, const m& b) { return a += b; }
4f4     friend m operator -(m a, const m& b) { return a -= b; }
4d5     friend m operator *(m a, const m& b) { return a *= b; }
2c4     friend m operator /(m a, const m& b) { return a /= b; }
3c9     friend m operator ^(m a, ll b) { return a ^= b; }
66d };
```

### 3.16 Linear Sieve (prime precomputation) bce

*Computes primes up to N in O(N) using the smallest prime factor (lp). pr will contain all primes <= N. lp[x] stores the smallest prime dividing x.*
**Complexity:** O(N)

```
998 pair<v64, v64> sieve(ll n){
96b     v64 lp(n+1);
87a     v64 pr;
1df     forn(i, 2, n+1) {
d68         if (lp[i] == 0) {
98a             lp[i] = i;
e0d             pr.push_back(i);
b04         }
d6c         for (ll j = 0; i * pr[j] <= n; ++j) {
97c             lp[i * pr[j]] = pr[j];
998             if (pr[j] == lp[i]) {
c9a                 break;
963             }
4da         }
57c     }
306     return {pr, lp};
c34 }

3c3 v64 calc_mu(v64& lp) {
429     ll n = sz(lp) - 1;
872     v64 ans(n);
375     ans[1] = 1;
a7c     forn(i,2,n) {
207         ll p = lp[i], x = i/p;
f5b         if (lp[x] == p) ans[i] = 0;
68b         else ans[i] = -ans[x];
f06     }
ada     return ans;
037 }

766 v64 calc_d(v64& lp) {
429     ll n = sz(lp) - 1;
872     v64 ans(n);
375     ans[1] = 1;
a7c     forn(i, 2, n) {
207         ll p = lp[i], x = i / p;
9d6         if (lp[x] != p) ans[i] = 1;
c39         else ans[i] = ans[x] + 1;
c7a     }
a7c     forn(i, 2, n) {
207         ll p = lp[i], x = i / p;
569         if (ans[i] == 1) ans[i] = ans[x] * 2;
a40         else ans[i] = (ans[x] / ans[i]) * (ans[i] + 1);
fa2     }
ada     return ans;
f7f }
```

# 4 String Algorithms

### 4.1 Aho-Corasick Automaton 01f

*Builds a trie with failure links for multi-pattern matching; insert is O(|s|), build is linear in total length, and queries run in linear time in the text.*
**Complexity:** varies, O(total patterns length)

```
427 namespace aho {
7bc     map<char, ll> to[MAX];
452     ll link[MAX], idx, term[MAX], exit[MAX], sobe[MAX];
db4     vector<ll> max_match(MAX, 0);

c0c     void insert(string& s) {
9a3         ll at = 0;
419         for (char c : s) {
5b1             auto it = to[at].find(c);
b64             if (it == to[at].end()) at = to[at][c] = ++idx;
b2d             else at = it->second;
44e         }
2ba         term[at]++, sobe[at]++;
5f8         max_match[at] = sz(s);
b7a     }

91f     void build() {
567         queue<ll> q;
b47         q.push(0);
478         link[0] = exit[0] = -1;
dad         while (sz(q)) {
950             ll i = q.front(); q.pop();
df1             for (auto [c, j] : to[i]) {
1bd                 ll l = link[i];
657                 while (l != -1 && !to[l].count(c)) l = link[l];
a68                 link[j] = l == -1 ? 0 : to[l][c];
546
                exit[j] = term[link[j]] ? link[j] : exit[link[j]];

02b                 max_match[j] = max(max_match[link[j]], max_match
[j]);
fb8                 if (exit[j]+1) sobe[j] += sobe[exit[j]];
df8                 q.push(j);
a97             }
e9c         }
555     }
3b5     ll query(string& s) {
a33         ll at = 0, ans = 0;
```

```
419         for (char c : s){
bae             while (at != -1 && !to[at].count(c)) at = link[at];
2ab             at = at == -1 ? 0 : to[at][c];
56a             ans += sobe[at];
deb         }
ada         return ans;
c01     }
078     vector<ll> match_vec(string& s) {
fec         ll at = 0, n = sz(s);
ccc         vector<ll> v(n, 0);
bda         forn(i, 0, n){
aa0             char c = s[i];
bae             while (at != -1 && !to[at].count(c)) at = link[at];
2ab             at = at == -1 ? 0 : to[at][c];

ab5             v[i] = max_match[at]; // quero isso
63e         }
f1e         return v;
7a7     }
01f }
```

## 4.2 Knuth-Morris-Pratt (KMP) 514

*Computes prefix function and performs linear-time substring search with optional automaton construction.*

**Complexity:** O(n + m), O(n)

```
7f8 v64 pi(string& s) {
761     v64 p(sz(s));
655     for (ll i = 1, j = 0; i < sz(s); i++) {
407         while (j && s[j] != s[i]) j = p[j-1];
1c1         if (s[j] == s[i]) j++;
6fb         p[i] = j;
314     }
4e5     return p;
3fb }

158 v64 match(string& pat, string& s) {
086     v64 p = pi(pat), match;
4e5     for (ll i = 0, j = 0; i < sz(s); i++) {
d77         while (j && pat[j] != s[i]) j = p[j-1];
c8b         if (pat[j] == s[i]) j++;
1b1         if (j == sz(pat)) match.push_back(i-j+1), j = p[j-1];
f18     }
de3     return match;
982 }

1c8 struct KMPaut : vector<v64> {
ff6     KMPaut(){}
29d     KMPaut (string& s) : vector<v64>(26, v64(sz(s)+1)) {
3f3         v64 p = pi(s);
b29         auto& aut = *this;
c31         aut[s[0]-'a'][0] = 1;
50a         for (char c = 0; c < 26; c++)
2bf             for (int i = 1; i <= sz(s); i++)
1a8
                aut[c][i] = s[i]-'a' == c ? i+1 : aut[c][p[i-1]];
4a6     }
3b4 };
```

## 4.3 Suffix Array - O(n log n) bfc

*kasai recebe o suffix array e calcula lcp[i], o lcp entre s[sa[i],...,n-1] e s[sa[i+1],...,n-1] Complexidades: suffix_array - O(n log(n)) kasai - O(n)*

```
c73 v64 suffix_array(string s) {
3c2     s.push_back('$'); // 0 caso v64 (CHECAR SE PODE)
0ce     ll n = sz(s), N = max(n, 260ll);
bb4     v64 sa(n), ra(n);
c15     forn(i, 0, n) sa[i] = i, ra[i] = s[i];

d0d     for(ll k = 0; k < n; k ? k *= 2 : k++) {
b35         v64 nsa(sa), nra(n), cnt(N);

dff         for(int i = 0; i < n; i++) nsa[i] = (nsa[i]-k+n)%n, cnt[
ra[i]]++;

20e         forn(i, 1, N) cnt[i] += cnt[i-1];
ece
        for(ll i = n-1; i+1; i--) sa[--cnt[ra[nsa[i]]]] = nsa[i];

034         for(ll i = 1, r = 0; i < n; i++) nra[sa[i]] = r += ra[sa
[i]] !=
ce2             ra[sa[i-1]] || ra[(sa[i]+k)%n] != ra[(sa[i-1]+k)%n];
b03         ra = nra;
666         if (ra[sa[n-1]] == n-1) break;
969     }
054     return v64(sa.begin()+1, sa.end());
69d }

7a6 v64 kasai(string s, v64 sa) {
645     ll n = sz(s), k = 0;
ce4     v64 ra(n), lcp(n);
9c3     forn(i, 0, n) ra[sa[i]] = i;

b9d     for (ll i = 0; i < n; i++, k -= !!k) {
c37         if (ra[i] == n-1) { k = 0; continue; }
8ba         ll j = sa[ra[i]+1];
443         while (i+k < n && j+k < n && s[i+k] == s[j+k]) k++;
840         lcp[ra[i]] = k;
bd0     }
8ad     return lcp;
60f }
```

## 4.4 Trie (Prefix Tree) 57a

*Stores strings over a fixed alphabet to support insert, erase, and prefix counting in linear time.*

**Complexity:** O(|s|) per op, O(total keys)

```
698 struct trie {
b3b     vector<v64> to;
2a7     v64 end, pref;
db7     ll sigma; char norm;

7b2     trie(ll sigma_=26, char norm_='a') : sigma(sigma_), norm(
norm_) {
0ba         to = {v64(sigma)};
0fe         end = {0}, pref = {0};
9b4     }

aa4     void insert(string s) {
bf5         ll x = 0;
2cf         for (auto c : s) {
fde             ll &nxt = to[x][c-norm];
dff             if (!nxt) {
f8e                 nxt = sz(to);
89b                 to.push_back(v64(sigma));
625                 end.push_back(0), pref.push_back(0);
04f             }
cc9             x = nxt, pref[x]++;
9a3         }
d74         end[x]++, pref[0]++;
9e2     }

827     void erase(string s) {
bf5         ll x = 0;
419         for (char c : s) {
fde             ll &nxt = to[x][c-norm];
6cf             x = nxt, pref[x]--;
7b3             if (!pref[x]) nxt = 0;
3c9         }
50b         end[x]--, pref[0]--;
593     }

9e6     ll find(string s) {
bf5         ll x = 0;
2cf         for (auto c : s) {
ed9             x = to[x][c-norm];
092             if (!x) return -1;
7d6         }
e44         return x;
1a5     }

2e0     ll count_pref(string s) {
47a         ll id = find(s);
a65         return id >= 0 ? pref[id] : 0;
08c     }
57a };
```

## 4.5 Z-funtion 84b

*For any index i, finds the lcp between s[i..n) ans s O(n)*

```
ffc v64 get_z(string s) {
f56     ll n = sz(s);
cb9     v64 z(n, 0);

897     ll l = 0, r = 0;
6ad     forn(i, 1, n) {
6be         if (i <= r) z[i] = min(r - i + 1, z[i - l]);
e44         while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
bfb         if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
3b9     }

ede     return z;
84b }
```

# 5 Dynamic Programming

## 5.1 Divide and Conquer DP Optimization 1b1

*Optimizes DP transitions with quadrangle inequality/monge-like structure using divide-and-conquer over optimal decision points.*

**Complexity:** O(K N log N) with O(1) cost, O(N)

```
4d6 vector<v64> dp; // dp[n+1][2]

afe void solve(ll k, ll l, ll r, ll lk, ll rk) {
bd4     if (l > r) return;
775     ll m = (l+r)/2, p = -1;
66f     auto& ans = dp[m][k&1] = INF;
c8a     for (ll i = max(m, lk); i <= rk; i++) {
389         ll at = dp[i+1][~k&1] + cost(m, i);
```

```
19b         if (at < ans) ans = at, p = i;
e3c     }
129     solve(k, l, m-1, lk, p), solve(k, m+1, r, p, rk);
c8f }

8c3 ll dnc(ll n, ll k) {
b45     dp[n][0] = dp[n][1] = 0;
925     forn(i,0,n) dp[i][0] = INF;
157     forn(i,1,k+1) solve(i, 0, n-i, 0, n-i);
b9c     return dp[0][k&1];
aa3 }
```

### 5.2 Sum over Subsets DP (SOS DP) 773

*sos_dp_sub: for each mask, computes the sum of f[s] over all subsets s of mask (i.e., f'[mask] = sum of f[s] for s in mask). sos_dp_super: for each mask, computes the sum of f[s] over all supersets s of mask (i.e., f'[mask] = sum of f[s] for mask in s). input size must be a power of two (2{}n).*

**Complexity:** O(n * 2{}n), O(2{}n) mem

```
66a v64 sos_dp_sub(v64 f) {
848     ll n = __builtin_ctzll(sz(f));
77f     assert((1ll<<n) == sz(f));

00d     forn(i,0,n) forn(mask, 0, (1<<n))
ec7         if (mask>>i&1) f[mask] += f[mask^(1<<i)];
         // -= for inverse
f41     return f;
e2d }

69d v64 sos_dp_super(v64 f) {
848     ll n = __builtin_ctzll(sz(f));
77f     assert((1ll<<n) == sz(f));

00d     forn(i,0,n) forn(mask, 0, (1<<n))
22b         if (~mask>>i&1) f[mask] += f[mask^(1<<i)];
         // -= for inverse
f41     return f;
be7 }
```

# 6 Computational Geometry

### 6.1 Circumcircle and Circumradius 1c2

*ccRadius: calcula o circumraio funciona com pt de int ccCentar: calcula o circumcentro funciona somente com pt de double*

```
4de ld ccRadius(pt a, pt b, pt c) {
558     ld s = sarea(a,b,c);
5c4     if(s == 0) return DINF;
34c     return dist(a,b)*dist(b,c)*dist(c,a)/(4*abs(s));
f63 }

7c1 pt ccCenter(pt a, pt b, pt c) {
8ae     c = c-a, b = b-a;
4f3     if((b^c) == 0) return {DINF,DINF};
fb9     return a + perp((b*norm_sq(c)-c*norm_sq(b)))/(2*(b^c));
97b }
```

### 6.2 Convex Hull (Monotone Chain) 23f

*Computes the convex hull of a set of points using Andrew's monotone chain; handles collinear points based on ccw condition.*

**Complexity:** O(N log N), O(N)

```
972 vector<pt> convex_hull(vector<pt> &v) {
cd9     v.erase(unique(v.begin(), v.end()), v.end());
6f2     if(sz(v) <=1) return v;
9aa     vector<pt> l, u;
e1f     forn(i, 0, sz(v)) {
986         while(sz(l) > 1 && !ccw(l.end()[-2], l.end()[-1], v[i]))
27f             l.pop_back();
482         l.push_back(v[i]);
a9a     }
e65     for(ll i = sz(v)-1; i>=0; i--) {
dfb         while(sz(u) > 1 && !ccw(u.end()[-2], u.end()[-1], v[i]))
74d             u.pop_back();
73b         u.push_back(v[i]);
d4e     }
27f     l.pop_back();
74d     u.pop_back();
106     for(pt i : u) l.push_back(i);
273     return l;
23f }
```

### 6.3 Extremal Vertex in Convex Polygon b8e

*Dado comparador, calcula o maior vertice nessa ordem O(log(n))*

```
40e #include "src/geometry/geo_int.cpp"

2c6 ll extreme(vector<pt>&pol, const function<bool(pt, pt)>& cmp) {
1c8     ll n = sz(pol);
3b2     auto extr = [&](ll i, bool& cur_dir) {
06c         cur_dir = cmp(pol[(i+1)%n], pol[i]);
b45         return !cur_dir && !cmp(pol[(i+n-1)%n], pol[i]);
eac     };
e9f     bool last_dir, cur_dir;
3cf     if (extr(0, last_dir)) return 0;
c6d     ll l = 0, r = n;
69d     while (l+1 < r) {
c61         ll m = (l+r)/2;
69a         if (extr(m, cur_dir)) return m;
fe2         bool rel_dir = cmp(pol[m], pol[l]);
df4         if ((!last_dir && cur_dir) || (last_dir == cur_dir &&
rel_dir == cur_dir)) {
fd9             l = m;
634             last_dir = cur_dir;
f0e         } else r = m;
79c     }
273     return l;
d8e }
e9e ll max_dot(vector<pt>&pol, pt v) {
f10     return extreme(pol, [&](pt p, pt q) { return p*v > q*v; });
844 }
b8d pair<ll, ll> tangents(vector<pt>&pol, pt p) {
60f     auto L = [&](pt q, pt r) { return ccw(p, r, q); };
36c     auto R = [&](pt q, pt r) { return ccw(p, q, r); };
9ef     return {extreme(pol, L), extreme(pol, R)};
7d7 }
```

### 6.4 Float Geometry Primitives e8d

*Defines 2D point and line structures with orientation, area, and angle comparisons plus a sweep-line comparator.*

```
17e typedef long double ld;

b47 const ld DINF = 1e18;
3b5 const ld eps = 1e-9;
45e const ld pi = 2*acos(0);

c6e #define sq(x) ((x)*(x))

1fc ll cmp(ld x, ld y = 0, ld tol = eps) {
534     return (x <= y + tol) ? (x + tol < y) ? -1 : 0 : 1;
49f }

c01 struct pt { // ponto
2d5     ld x, y;
292     pt(ld x_ = 0, ld y_ = 0) : x(x_), y(y_) {}
537     bool operator < (const pt p) const {
ccf         if (cmp(x, p.x) != 0) return cmp(x, p.x) < 0;
6c8         if (cmp(y, p.y) != 0) return cmp(y, p.y) < 0;
ca4         return 0;
554     }
652     bool operator == (const pt p) const {
580         return cmp(x, p.x) == 0 && cmp(y, p.y) == 0;
6df     }
377
    pt operator + (const pt p) const { return pt(x+p.x, y+p.y); }
e45
    pt operator - (const pt p) const { return pt(x-p.x, y-p.y); }
4e0
    pt operator * (const ld c) const { return pt(x*c  , y*c  ); }
3aa
    pt operator / (const ld c) const { return pt(x/c  , y/c  ); }
1c3     ld operator * (const pt p) const { return x*p.x + y*p.y; }
59b     ld operator ^ (const pt p) const { return x*p.y - y*p.x; }
97f     friend istream& operator >> (istream& in, pt& p) {
7cf         return in >> p.x >> p.y;
3a6     }
759 };

e42 struct line { // reta
ca5     pt p, q;
9d3     line() {}
9b5     line(pt p_, pt q_) : p(p_), q(q_) {}
410     friend istream& operator >> (istream& in, line& r) {
def         return in >> r.p >> r.q;
2c5     }
4f9 };

4a5 ld norm_sq(pt p){ return p*p;}

857 ld norm(pt p){ return sqrt(norm_sq(p)); }

8cb ld dist_sq(pt p, pt q){ return norm_sq(p - q);}

3ea ld dist(pt p,pt q){ return sqrt(dist_sq(p, q));}

3c7 ld angle(pt v) { // angulo do vetor com o eixo x
d57     ld ang = atan2(v.y, v.x);
c65     if (cmp(ang, 0) < 0) ang += 2*pi;
dbd     return ang;
b17 }

beb ld sarea(pt p, pt q, pt r) { // area com sinal
f6e     return ((q-p)^(r-q))/2;
9cd }

093 bool col(pt p, pt q, pt r) { // se p, q e r sao colin.
f02     return cmp(sarea(p, q, r), 0) == 0;
```

```
495 }

643 bool ccw(pt p, pt q, pt r, bool strict = true) { // true if p, q, r are
a7a     return cmp(sarea(p, q, r)) > 0 || (!strict && cmp(sarea(p, q, r)) == 0);
44e }

ae5 bool isinseg(pt p, line r) { // se p pertence ao seg de r
ee2     pt a = r.p - p, b = r.q - p;
c52     return cmp(a ^ b) == 0 && cmp(a * b) <= 0;
9ff }

a4d ld polarea(vector<pt> v) { // area do poligono
5d8     ld ret = 0;
2d0     forn(i, 0, sz(v))
65d         ret += sarea(pt(0, 0), v[i], v[(i + 1) % sz(v)]);
ba7     return abs(ret);
d71 }

4f8
ld get_t(pt v, line r) { // retorna t tal que t*v pertence a reta r
22e     return (r.p^r.q) / ((r.p-r.q)^v);
114 }

9d5 pt inter(line r, line s) { // r inter s
a3d     if (cmp((r.p - r.q) ^ (s.p - s.q), 0) == 0) return pt(DINF, DINF);
7c8     r.q = r.q - r.p, s.p = s.p - r.p, s.q = s.q - r.p;
d1a     return r.q * get_t(r.q, s) + r.p;
50b }

// Intersecções de circulos

d3a pair<pt,pt> perp_pts(line l, ld d){ // retorna dois pontos r,s tq rq e sq sao perp a pq e |rq| = |sq| = d;
22c     pt dir = l.p - l.q;
f3e     pt r = pt(dir.y, -dir.x);
501     pt k = r * (d/norm(r));
926     return {l.q + k, l.q - k};
63c }

827 pair<pt,pt> circint(pt o1, ld r1, pt o2, ld r2){
b73     pt dir = o2 - o1;
ea2     pt in(DINF, DINF);
5e2     ld d = norm(dir);
    // tangent
58b     if(cmp(r1 + r2, d) == 0 || cmp(d + r2, r1) == 0 || cmp(d + r1, r2) == 0)
a45         return {o1 + dir * r1/(r1 + r2), in};
    // no inter
58c     if(cmp(r1 + r2, d) == -1 || cmp(d + r2, r1) == -1 || cmp(d + r1, r2) == -1)
e03         return {in, in};
    // two inters
98f     ld x = (d*d - r2*r2 + r1*r1)/(2*d);
cfb
    return perp_pts(line(o1, o1 + dir * x/d), sqrt(r1*r1 - x*x));
b02 }

//interseções de circulo e reta

b05 pt foot(pt p, pt a, pt b) {
fec     pt dir = b-a;
d67     return (dir*((p-a)*dir))/(dir*dir) + a;
bba }

db0 pair<pt,pt> cline(pt o, double r, pt a, pt b){
2d2     pt h = foot(o, a, b);
ea2     pt in(DINF, DINF);
38c     ld d = norm(h - o);
227     if(cmp(d, r) == 0) return {h, in};
51e     if(cmp(d, r) == 1) return {in, in};
3b7     return perp_pts(line(o, h), sqrt(r*r - d*d));
8d7 }
```

## 6.5 Integer Geometry Primitives a52

*Defines 2D point and line structures with orientation, area, and angle comparisons plus a sweep-line comparator.*

```
c01 struct pt { // point
57c     ll x, y;
8cc     pt(ll x_ = 0, ll y_ = 0) : x(x_), y(y_) {}
537     bool operator < (const pt p) const {
dac         if (x != p.x) return x < p.x;
2d1         return y < p.y;
f98     }
652     bool operator == (const pt p) const {
4f6         return x == p.x && y == p.y;
c9e     }
377
    pt operator + (const pt p) const { return pt(x+p.x, y+p.y); }
e45
    pt operator - (const pt p) const { return pt(x-p.x, y-p.y); }
953     pt operator * (const ll c) const { return pt(x*c, y*c); }
881     ll operator * (const pt p) const { return x*(ll)p.x + y*(ll)p.y; } // dot product
f4f     ll operator ^ (const pt p) const { return x*(ll)p.y - y*(ll)p.x; } // cross product
97f     friend istream& operator >> (istream& in, pt& p) {
7cf         return in >> p.x >> p.y;
3a6     }
d91 };

e42 struct line { // line segment
ca5     pt p, q;

9d3     line() {}
9b5     line(pt p_, pt q_) : p(p_), q(q_) {}
fe1     bool operator < (const line l) const {
d8d         if (!(p == l.p)) return p < l.p;
46b         return q < l.q;
887     }
935     bool operator == (const line l) const {
c63         return p == l.p && q == l.q;
485     }
410     friend istream& operator >> (istream& in, line& r) {
def         return in >> r.p >> r.q;
2c5     }
ead };

c60 ll sarea2(pt p, pt q, pt r) { // 2 * signed area
411     return (q-p)^(r-q);
74b }

9ad ll norm_sq(pt p){ return p*p;}
5b9 ll dist_sq(pt p, pt q){ return norm_sq(p - q);}

643 bool ccw(pt p, pt q, pt r, bool strict = true) {
c35
    return sarea2(p, q, r) > 0 || (!strict && sarea2(p, q, r) == 0);
bc3 }

b9f int quad(pt p) { // quadrant of a point
472     return (p.x<0)^3*(p.y<0);
09a }

6f3 bool compare_angle(pt p, pt q) { // returns true if angle(p) < angle(q)
591     if (quad(p) != quad(q)) return quad(p) < quad(q);
74f     return ccw(q, pt(0, 0), p);
b1c }

ae5 bool isinseg(pt p, line r) { // se p pertence ao seg de r
ee2     pt a = r.p - p, b = r.q - p;
c3c     return (a ^ b) == 0 && (a * b) <= 0;
7c8 }

a74 bool interseg(line r, line s) { // se o seg de r intersecta o seg de s
8dd     if (isinseg(r.p, s) || isinseg(r.q, s)
d30         || isinseg(s.p, r) || isinseg(s.q, r)) return 1;

4b8     return ccw(r.p, r.q, s.p) != ccw(r.p, r.q, s.q) &&
7a3             ccw(s.p, s.q, r.p) != ccw(s.p, s.q, r.q);
1be }

806 ll polarea(vector<pt> v) { // area do poligono
125     ll ret = 0;
2d0     forn(i, 0, sz(v))
49f         ret += sarea2(pt(0, 0), v[i], v[(i + 1) % sz(v)]);
ba7     return abs(ret);
80a }

// comparator for set to perform sweep line with segments
ca2 struct cmp_sweepline {
553     bool operator () (const line& a, const line& b) const {
        // assumes segments have p < q
f8c         if (a.p == b.p) return ccw(a.p, a.q, b.q);
72e         if (a.p.x != a.q.x && (b.p.x == b.q.x || a.p.x < b.p.x))
b87             return ccw(a.p, a.q, b.p);
c35         return ccw(a.p, b.q, b.p);
cae     }
fc1 };
```

## 6.6 Halfplane Intersection 392

*Acha o ponto mais extremo na direção dada só entra retas nao verticais, processar as retas verticais antes e encodar essa info no minx e maxx se quiser o ponto mais acima, fazer best_dir = (0,1) dependendo do best_dir, best = (+-INF, +-INF)*

```
7b5 mt19937_64 rng((ll) chrono::steady_clock::now().time_since_epoch().count());

fa6 pt hpi_1d(vector<line> v, pt best_dir, ll n, line suport, ll minx, ll maxx){
020     pt low = inter(line(pt(minx, 0), pt(minx,1)), suport), upp = inter(line(pt(maxx, 0), pt(maxx,1)), suport);
bda     forn(i, 0, n){
```

```
331         pt intersection = inter(v[i], suport);
f09         debug(low);
3fc         debug(upp);
801         debug(intersection);
ebe         debug(v[i]);

bee         if(!ccw(v[i].p, v[i].q, low)) low = intersection;
d66         else if(!ccw(v[i].p, v[i].q, upp)) upp = intersection;
5bd     }

321     if(cmp(low.x, upp.x) > 0) return {DINF, DINF};

bcd     if(cmp(best_dir*low, best_dir*upp) <= 0) return upp;
c60     return low;
82a }

d04 pt hpi(vector<line> v, pt best_dir, ll minx, ll maxx){
efa     shuffle(v.begin(), v.end(), rng);

3d3     ll n = sz(v);
559     pt best(1, 1);

bda     forn(i, 0, n){
54d         debug(best);
ebe         debug(v[i]);
d42         if(ccw(v[i].p, v[i].q, best)) continue;
894         best = hpi_1d(v, best_dir, i, v[i], minx, maxx);
a9f         assert(!(best ==pt(DINF, DINF)));
269     }

5cb     return best;
993 }
```

## 6.7 Point Inside Polygon 11a

*responde se um ponto está dentro do poligono com ray casting se strict = true ele nao considera pontos na borda como interior O(n)*

```
ffe #include "src/geometry/geo_double.cpp"

7f8 bool inpolygon(vector<pt> &p, pt a, bool strict = true) {
cf5     ll cnt = 0, n = sz(p);
bda     forn(i,0,n) {
3e6         pt q = p[(i + 1) % n];
5bd         if (isinseg(a, line(p[i], q))) return !strict;
d9f         cnt ^= (((cmp(a.y, p[i].y) < 0) - (cmp(a.y,q.y) < 0))*((
p[i]-a)^(q-a))) > 0 ;
e16     }
fa7     return cnt;
618 }
```

## 6.8 Point Inside Convex Polygon 4a9

*responde se um ponto está dentro do poligono convexo com binary search se strict = true ele nao considera pontos na borda como interior O(log(n))*

```
40e #include "src/geometry/geo_int.cpp"

314 bool inpolygonconv(vector<pt> &pol, pt p, bool strict = true) {
123     if (pol.size() == 0) return false;
dc0     if (pol.size() == 1) return (!strict && p == pol[0]);
2e0     ll l = 1, r = pol.size();
bec     while (l < r) {
c61         ll m = (l+r)/2;
198         if (ccw(p, pol[0], pol[m])) l = m+1;
3cf         else r = m;
f42     }
434     if (l == 1) return (!strict && isinseg(p, line(pol[0], pol
[1])));
612     if (l == pol.size()) return false;
567     return !ccw(p, pol[l], pol[l-1], !strict);
700 }
```

## 6.9 Minimum Enclosing Circle 2a8

*calcula o minimum enclosing circle, probalistico mas e O(n) so se o pt for de double!! talvez usar r*(1+eps);*

```
01a pair<pt, ld> mec(vector<pt> v) {
57d     shuffle(v.begin(), v.end(), mt19937(time(0)));
0a0     pt o = v[0];
33e     ld r = 0;
baf     forn(i, 0, sz(v)) if (cmp(dist(o, v[i]),r) > 0) {
875         o = v[i], r = 0;
6c7         forn(j, 0, i) if (cmp(dist(o, v[j]),r) > 0) {
297             o = (v[i] + v[j]) / 2;
735             r = dist(o, v[i]);
799             forn(k, 0, j) if (cmp(dist(o, v[k]),r) > 0) {
7ed                 o = ccCenter(v[i], v[j], v[k]);
735                 r = dist(o, v[i]);
955             }
78d         }
096     }
571     return {o, r};
2a8 }
```

## 6.10 Minkowski Sum 374

*Soma poligonos de tamanho a e b em O(a+b)*

```
40e #include "src/geometry/geo_int.cpp"

18a vector<pt> minkowski(vector<pt> p, vector<pt> q) {
772     auto fix = [](vector<pt>& P) {
c3d
        rotate(P.begin(), min_element(P.begin(), P.end()), P.end());
a7c         P.push_back(P[0]), P.push_back(P[1]);
cd9     };
87e     fix(p), fix(q);
abe     vector<pt> ret;
aa2     int i = 0, j = 0;
e52     while (i < p.size()-2 or j < q.size()-2) {
f30         ret.push_back(p[i] + q[j]);
b77         auto c = ((p[i+1] - p[i]) ^ (q[j+1] - q[j]));
c73         if (c >= 0) i = min<int>(i+1, p.size()-2);
10d         if (c <= 0) j = min<int>(j+1, q.size()-2);
ea8     }
3cb     return ret;
f15 }
```

## 6.11 Plota Poligonos 73d

*Ajuda a debugar codigo de geometria*

```
def svg(P, o="d.svg"):
    if not P: return
    X, Y = zip(*P)
    x0, x1, y0, y1 = min(X), max(X), min(Y), max(Y)
    p = max(x1-x0, y1-y0) * 0.1 or 10
    head = f'<svg viewBox="{x0-p} {y0-p} {x1-x0+2*p} {y1-y0+2*p}"
xmlns="http://www.w3.org/2000/svg" style="background:#eee">'
    poly = f'<polygon points="{" ".join(f"{x},{y}" for x,y in P)}"
fill="#f003" stroke="red" stroke-width="{p/20}"/>'
    text = "".join([f'<text x="{x}" y="{y}" font-size="{p/3}">{i}</
text>' for i,(x,y) in enumerate(P)])
    with open(o, 'w') as f: f.write(head + poly + text + '</svg>')
```

## 6.12 Polygon Cut 01f

*Corta um poligono deixando somente a parte ccw com a reta l dada O(n)*

```
a43 vector<pt> polygonCut(const vector<pt>& poly, line l) {
ada     vector<pt> res;
524     forn(i,0,sz(poly)) {
ca2         pt curr = poly[i], prev = i ? poly[i-1] : poly.back();
c20         bool a = ccw(l.p, l.q, curr), b = ccw(l.p, l.q, prev);
0f7         if (a != b)
2b2             res.push_back(inter(line(curr, prev), l));
bcd         if (a)
249             res.push_back(curr);
8c3     }
8df     return res;
01f }
```

## 6.13 Segment Sweep Line Skeleton 15e

*Maintains an active set of segments ordered for sweep-line processing over x; insertion and removal are typically logarithmic.*

```
3fc map<ll, set<line, cmp_sweepline>> sweepline_begin; // dado um x,
 diz quais linhas comecam naquele x
d9f map<ll, set<line, cmp_sweepline>> sweepline_end;   // dado um x,
 diz quais linhas terminam naquele x

fbc void process_beg(set<line, cmp_sweepline>& v, set<line,
cmp_sweepline>& active_line, vector<ll>& parent){
a6e     for(auto x : v){
e4d         active_line.insert(x);
         // processar uma linha que esta sendo adicionada
6f5     }
abe }

806 void process_end(set<line, cmp_sweepline>& v, set<line,
cmp_sweepline>& active_line){
a6e     for(auto x : v){
b64         active_line.erase(x);
7b3     }
4eb }

ef3 void sweepline(ll n){
608     set<line, cmp_sweepline> active_line;

739     while(!sweepline_begin.empty() || !sweepline_end.empty()){
782         auto it_beg = sweepline_begin.begin();
79b         auto it_end = sweepline_end.begin();

3fe         if(sweepline_end.empty()){
8d2             process_beg(it_beg->second, active_line, parent);
1d6             sweepline_begin.erase(it_beg);
426             continue;
815         }

9b4         if(sweepline_begin.empty() || it_end->first <= it_beg->
first){
a93             process_end(it_end->second, active_line);
e34             sweepline_end.erase(it_end);
```

```
426             continue;
7ad         }

8d2         process_beg(it_beg->second, active_line, parent);
1d6         sweepline_begin.erase(it_beg);
93e     }
e24 }
```

# 7 Extra (Templates and Utilities)

## 7.1 1D Coordinate Compression 88a

*Maps large or sparse values to a dense range [0, N-1]. Uses sort, unique, and lower_bound for fast mapping. Time Complexity: O(N log N) | Space Complexity: O(N)*

```
65c void compress(){
edb     v64 vals;
7d5     sort(vals.begin(), vals.end());
d52     vals.erase(unique(vals.begin(), vals.end()), vals.end());

de8     auto compress = [&](ll x) {
8a0         return (ll)(lower_bound(vals.begin(), vals.end(), x) -
vals.begin());
3e4     };
88a }
```

## 7.2 hash d78

```
sed -n $2','$3' p' $1 | sed '/^#w/d' | cpp -dD -P -fpreprocessed |
tr -d '[:space:]' | md5sum | cut -c-6
```

## 7.3 makefile 885

```
CXXFLAGS = -fsanitize=address,undefined -fno-omit-frame-pointer -g -
Wall -Wshadow -std=c++20 -Wno-unused-result -Wno-sign-compare
```

## 7.4 Pragmas acb

```
481 #pragma GCC optimize("O3,unroll-loops,fast-math")
941 #pragma GCC optimize("Ofast,fast-math,unroll-loops,inline")
c19 #pragma GCC optimize("Os")
```

## 7.5 random 7e7

```
7b5 mt19937_64 rng((ll) chrono::steady_clock::now().time_since_epoch
().count());

d10 ll uniform(ll l, ll r){
367     uniform_int_distribution<ll> uid(l, r);
2f9     return uid(rng);
988 }
```

## 7.6 stress d5d

```
P=a
make ${P} ${P}2 gen || exit 1
for ((i = 1; ; i++)) do
    ./gen $i > tin
    ./${P} < tin > tout
    ./${P}2 < tin > tout2
    if (! cmp -s tout tout2) then
        echo "--> entrada:"
        cat tin
        echo "--> saida1:"
        cat tout
        echo "--> saida2:"
        cat tout2
        break;
    fi
    echo $i
done
```

## 7.7 template be5

```
e39 #include <bits/stdc++.h>
23e using namespace std;

ab0 typedef long long ll;
9a1 typedef pair<ll, ll> p64;
609 typedef vector<ll> v64;

3e7 #define forn(i, s, e) for (ll i = (s); i < (e); i++)
196 #define sz(u) ((ll) u.size())
e18 #define ln "\n"

ba0 #ifdef DEBUG
4f6 #define trace(u) u
5cb #define _
8c1 #else
073 #define trace(u)
358 #define _ ios::sync_with_stdio(0); cin.tie(0)
f2e #endif

bf4 #define debug(u) trace(cout << #u " = " << u << ln)
ca0 #define debugv(v) trace(cout << #v ": "; for (auto xx : v) cout
<< xx << " "; cout << ln)

4e7 const ll INF = 0x3f3f3f3f3f3f3f3fll;

 // int main() {
 //     _;
 //     return 0;
// }
```

## 7.8 tester 155

```
P=a
make "$P" || exit 1

for i in {1..9}; do
    ./"$P" < "in/${P}${i}" > out_ &&
    cmp -s out_ "out/${P}${i}" &&
    echo "OK $i" ||
    { echo "NOT $i"; break; }
done
```

# 1 Counting Problems

## 1.1 Burnside's Lemma

Let $G$ be a group that acts on a set $X$. The Burnside Lemma states that the number of distinct orbits is equal to the average number of points fixed by an element of G.

$$T = \frac{1}{|G|} \sum_{g \in G} |\texttt{fix}(g)|$$

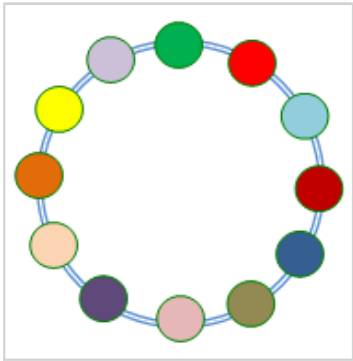
Where a orbit $\texttt{orb}(x)$ is defined as

$$\texttt{orb}(x) = \{y \in X : \exists g \in G \; gx = y\}$$

and $\texttt{fix}(g)$ is the set of elements in $X$ fixed by $g$

$$\texttt{fix}(g) = \{x \in X : gx = x\}$$

**Example:** With $k$ distinct types of beads how many distinct necklaces of size $n$ can be made? Considering that two necklaces are equal if the rotation of one gives the other.



$$T = \frac{1}{n} \sum_{i=0}^{n-1} k^{gcd(i,n)}$$

## 1.2 Catalan Numbers

1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, 208012, 742900, 2674440, 9694845, 35357670, 129644790, 477638700, 1767263190, 6564120420, 24466267020, 91482563640, 343059613650, 1289904147324, 4861946401452, 18367353072152, 69533550916004, 263747951750360, 1002242216651368.

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \frac{(2n)!}{(n+1)!n!} = \prod_{k=2}^{n} \frac{n+k}{k}, \quad n \geq 0$$

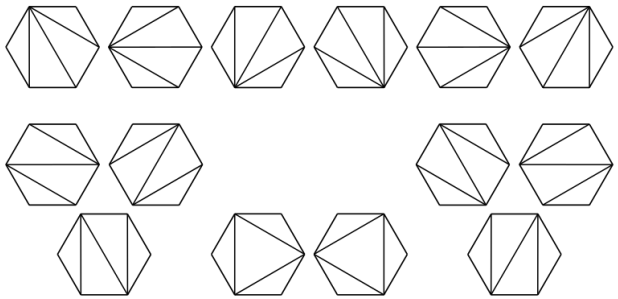
Applications:

- $C_n$ counts the number of expressions containing n pairs of parentheses which are correctly matched.

  (((())) ()(()) ()()() (())() ...

- Successive applications of a binary operator can be represented in terms of a full binary tree. (A rooted binary tree is full if every vertex has either two children or no children.) It follows that $C_n$ is the number of full binary trees with n + 1 leaves:

- $C_n$ is the number of different ways a convex polygon with n + 2 sides can be cut into triangles by connecting vertices with straight lines (a form of Polygon triangulation). The following hexagons illustrate the case n = 4:



# 2 Graphs

## 2.1 Sum of Subtrees of a Tree

For a rooted tree $T$ with $n$ vertices, let $sz(v)$ be the size of the subtree of $v$. Then the following holds:

$$\sum_{v \in V} \left[ \mathrm{sz}(v) + \sum_{u \text{ child of } v} \mathrm{sz}(u)(\mathrm{sz}(v) - \mathrm{sz}(u)) \right] = n^2$$

# 3 Number Theory

## 3.1 Chicken McNugget Theorem

Given two **coprime** numbers $n, m$, the largest number that cannot be written as a linear combination of them is $nm - n - m$.

- There are $\frac{(n-1)(m-1)}{2}$ non-negative integers that cannot be written as a linear combination of $n$ and $m$;
- For each pair $(k, nm - n - m - k)$, for $k \geq 0$, exactly one can be written.

## 3.2 Identities

$$\sum_{d|n} \varphi(d) = n$$

$$\sum_{\substack{i<n \\ \gcd(i,n)=1}} i = n \cdot \frac{\varphi(n)}{2}$$

$$|\{(x,y) : 1 \leq x, y \leq n, \gcd(x,y) = 1\}| = \sum_{d=1}^{n} \mu(d) \left\lfloor \frac{n}{d} \right\rfloor^2$$

$$\sum_{x=1}^{n} \sum_{y=1}^{n} \gcd(x,y) = \sum_{k=1}^{n} k \sum_{k|l}^{n} \left\lfloor \frac{n}{l} \right\rfloor^2 \mu\left(\frac{l}{k}\right)$$

$$\sum_{x=1}^{n} \sum_{y=x}^{n} \gcd(x,y) = \sum_{g=1}^{n} \sum_{g|d}^{n} g \cdot \varphi\left(\frac{d}{g}\right)$$

$$\sum_{x=1}^{n} \sum_{y=1}^{n} \mathrm{lcm}(x,y) = \sum_{d=1}^{n} d\,\mu(d) \sum_{d|l}^{n} l \binom{\lfloor \frac{n}{l} \rfloor + 1}{2}^2$$

$$\sum_{x=1}^{n} \sum_{y=x+1}^{n} \mathrm{lcm}(x,y) = \sum_{g=1}^{n} \sum_{g|d}^{n} d \cdot \varphi\left(\frac{d}{g}\right) \cdot \frac{d}{g} \cdot \frac{1}{2}$$

$$\sum_{x \in A} \sum_{y \in A} \gcd(x,y) = \sum_{t=1}^{n} \left( \sum_{l|t} \frac{t}{l} \mu(l) \right) \left( \sum_{t|a} \mathrm{freq}[a] \right)^2$$

$$\sum_{x \in A} \sum_{y \in A} \mathrm{lcm}(x,y) = \sum_{t=1}^{n} \left( \sum_{l|t} \frac{l}{t} \mu(l) \right) \left( \sum_{a \in A,\, t|a} a \right)^2$$

## 3.3 Large Prime Gaps

For numbers until $10^9$ the largest gap is 400.
For numbers until $10^{18}$ the largest gap is 1500.

## 3.4 Lucas's Theorem

$$\binom{n}{m} \equiv \prod_{i=0}^{k} \binom{n_i}{m_i} \pmod{p}$$

For $p$ prime. $n_i$ and $m_i$ are the coefficients of the representations of $n$ and $m$ in base $p$.

## 3.5 Mobius Inversion

If $g(n) = \sum_{d|n} f(d)$, then $f(n) = \sum_{d|n} g(d)\mu(\frac{n}{d})$.
A more useful definition is: $\sum_{d|n} \mu(d) = [n = 1]$
Example, sum of LCM:

$$\begin{aligned}
\sum_{i=1}^{n} \sum_{j=1}^{n} \mathrm{lcm}(i,j) &= \sum_{k=1}^{n} \sum_{i=1}^{n} \sum_{j=1}^{n} [\gcd(i,j) = k] \frac{ij}{k} \\
&= \sum_{k=1}^{n} \sum_{a=1}^{\lfloor \frac{n}{k} \rfloor} \sum_{b=1}^{\lfloor \frac{n}{k} \rfloor} [\gcd(a,b) = 1] abk \\
&= \sum_{k=1}^{n} k \sum_{a=1}^{\lfloor \frac{n}{k} \rfloor} a \sum_{b=1}^{\lfloor \frac{n}{k} \rfloor} b \sum_{d=1}^{\lfloor \frac{n}{k} \rfloor} [d|a]\,[d|b]\,\mu(d) \\
&= \sum_{k=1}^{n} k \sum_{d=1}^{\lfloor \frac{n}{k} \rfloor} \mu(d) \left( \sum_{a=1}^{\lfloor \frac{n}{k} \rfloor} [d|a]\, a \right) \left( \sum_{b=1}^{\lfloor \frac{n}{k} \rfloor} [d|b]\, b \right) \\
&= \sum_{k=1}^{n} k \sum_{d=1}^{\lfloor \frac{n}{k} \rfloor} \mu(d) \left( \sum_{p=1}^{\lfloor \frac{n}{kd} \rfloor} p \right) \left( \sum_{q=1}^{\lfloor \frac{n}{kd} \rfloor} q \right)
\end{aligned}$$

## 3.6 Number of Divisors

| $n$ | 6 | 60 | 360 | 5040 | 55440 | 720720 | 4324320 | 21621600 |
|---|---|---|---|---|---|---|---|---|
| $d(n)$ | 4 | 12 | 24 | 60 | 120 | 240 | 384 | 576 |

| $n$ | 367567200 | 6983776800 | 13967553600 | 321253732800 |
|---|---|---|---|---|
| $d(n)$ | 1152 | 2304 | 2688 | 5376 |

$184010559381256608 00 \approx 2e18$ is highly composite with 184320 divisors.
For numbers up to $10^{88}$, $d(n) < 3.6\sqrt[3]{n}$.

## 3.7 Prime Counting Function - $\pi(x)$

The prime counting function is asymptotic to $\frac{x}{\log x}$, by the prime number theorem.

| x | 10 | $10^2$ | $10^3$ | $10^4$ | $10^5$ | $10^6$ | $10^7$ | $10^8$ |
|---|---|---|---|---|---|---|---|---|
| $\pi(x)$ | 4 | 25 | 168 | 1 229 | 9 592 | 78 498 | 664 579 | 5 761 455 |

## 3.8 Some Primes

999999937 1000000007 1000000009 1000000021 1000000033
$10^{18} - 11$ $10^{18} + 3$ $2305843009213693951 = 2^{61} - 1$